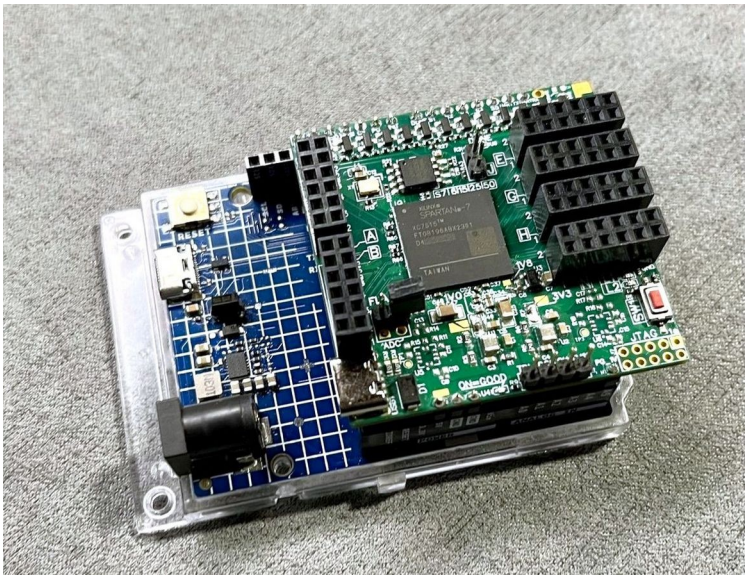
Quick Start Guide

&

Guide CLI ARDUINO® for the l MODULO FPGA SPARTAN7



Index

Chapter 1
Introduction

The "MODULO FPGA SPARTAN7" is a development board equipped with an FPGA and an PI NOR FLAHS (SNOR) memory.

The module can be use in two modes:

- **In combination with Arduino R4**, which acts as a programmer and management interface, etc.
- **In stand-alone mode**, by loading the FPGA configuration file into the SNOR memory and powering the module via its USB-C port, without the need for Arduino.

As we know, the FPGA does not have non-volatile memory in which the “configuration file” can be written; it only has volatile memory, so an external memory is required to store the configuration file. This external memory is the SNOR FLASH.

The Arduino R4 is used for:

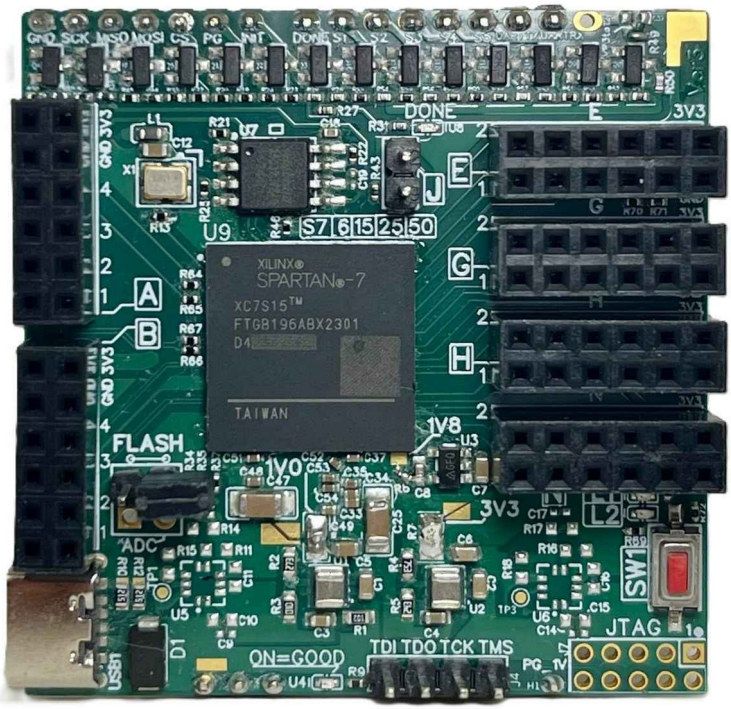
- Configuring the FPGA by loading the bitstream file directly (configuration is lost on reset).
- Writing the FPGA binary configuration file into the SNOR memory, thus enabling stand-alone operation of the module.
- Communicating with the FPGA through the **UART interface**.
- Monitoring the FPGA status and performing reset operations.
- Interfacing with 5 dedicated 5V I/O lines (S1–S5), mapped as follows:
    - S1 → PIN2
    - S2 → PIN3
    - S3 → PIN4
    - S4 → PIN5
    - S5 → PIN6

A dedicated software is uploaded to the Arduino R4 for managing the FPGA module.

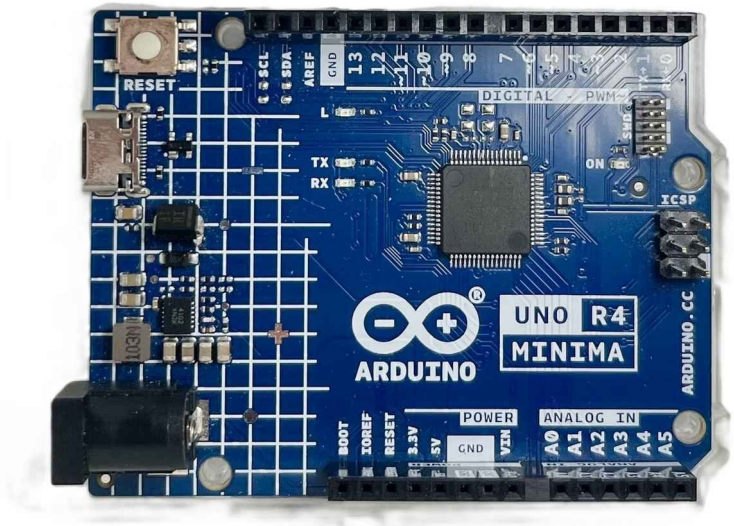
The purpose of this document is to describe the operation of this software, which provides the user with a command line interface (CLI) accessible via the UART-USB connection of the Arduino R4.

What You Need:

- MODULO FPGA SPARTAN7
  [PCB Version=3][min Rev00 see 4.2]
  [Block diagram see 4.5]



- Arduino UNO R4 Minima or Arduino UNO R4 WiFi



- Arduino® IDE 2.3.0 :
  https://www.arduino.cc/en/software/

- Arduino Software [FPGA_Schield-cli.ino Ver 0.0]
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/software/FPGA_Shield-cli.ino

- Tera Term [Version min 5.4.0]
  https://teratermproject.github.io/
  https://github.com/TeraTermProject/teraterm/releases

- Binary file or FPGA bitstream Xilinx SPARTAN 7.

- Pinout file of the "MODULO FPGA SPARTAN7" :
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/pinout/pinout_ver003_rev00.xdc

- Vivado (See how to install Vivado in 4.4) [Referance version 2024.2]

- USB-C cable (>=USB 2.0).

This project is not affiliated with or endorsed by Arduino.

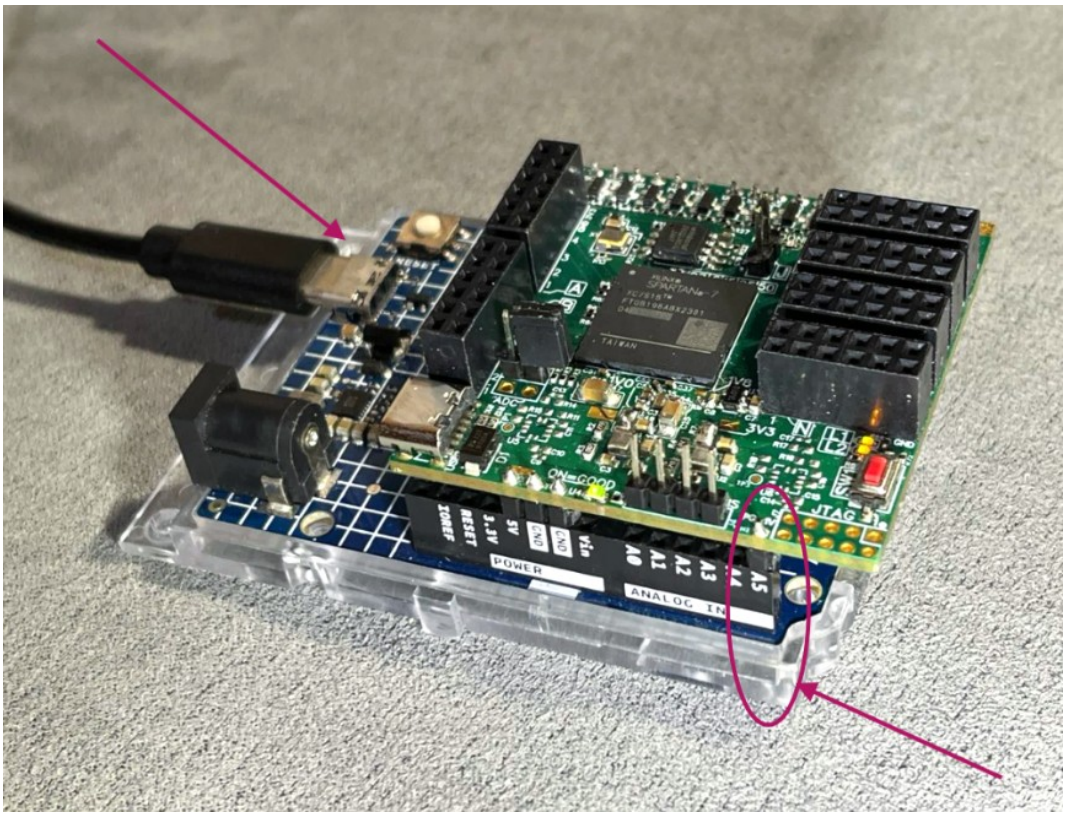
Chapter 2
First Startup

First, we will look at the software used to communicate with the "MODULO FPGA SPARTAN7" dobbiamo scaricare il software dalla repository presente su https://github.com/ e scaricarla nella scheda Arduino R4.

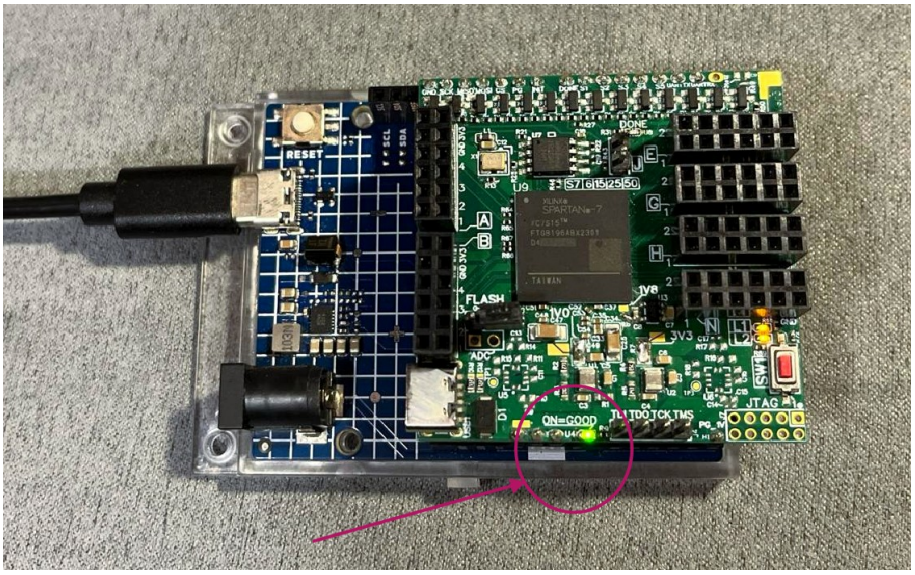
- Downaload the file: *https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/software/FPGA_Shield-cli.ino*

- Connect the "MODULO FPGA SPARTAN7" to Arduino, then connect Arduino's USB-C port to the PC.

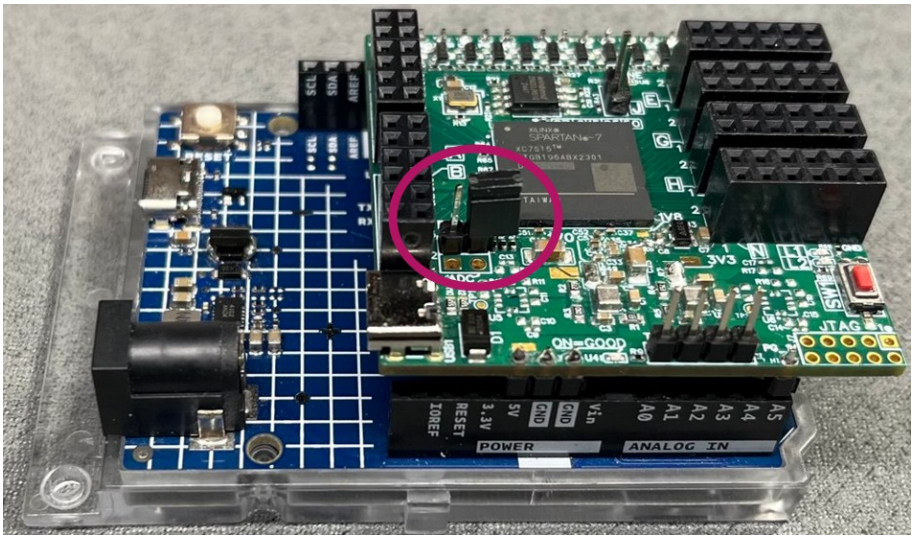
⚠ Attention: insert the FPGA Module into Arduino **without powering through USB-C**, to prevent short circuits during insertion of the shield.
Make sure to align the FPGA Module correctly: the **PG pin** of the "FPGA SPARTAN7 MODULE" must be connected to pin **A5** of Arduino.



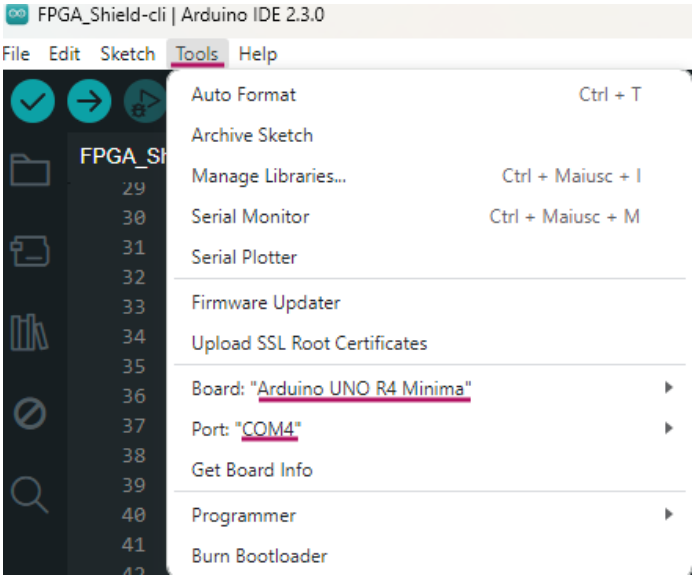
- If the connection is successful and the module is powered correctly, the U4 LED will be on.



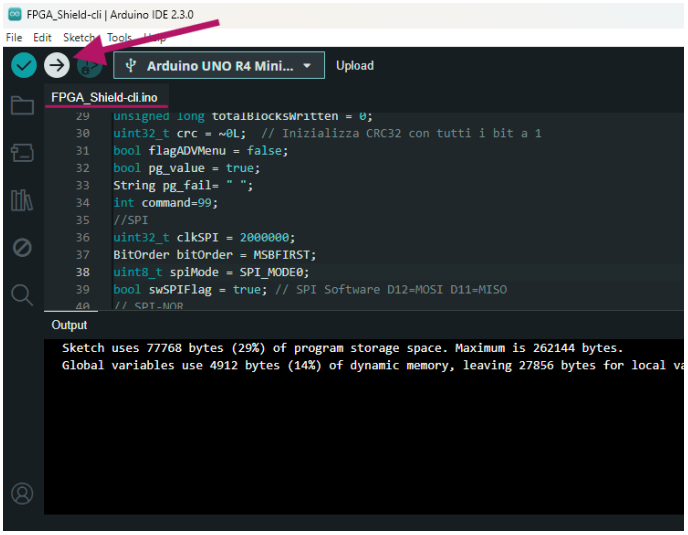
- Open the jumper. (Set the FPGA in Slave-SPI mode):



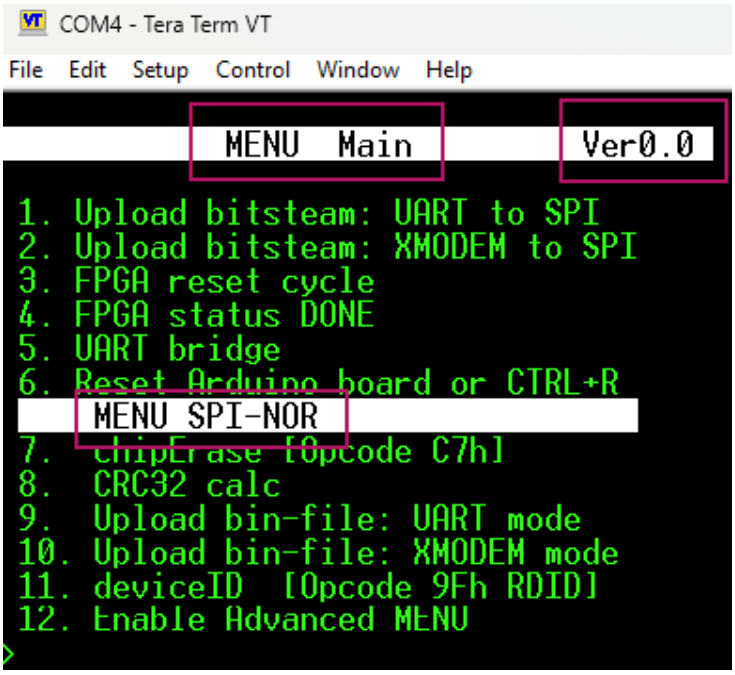
- Open the Arduino **IDE** and configure the correct **COMx** and board Type:



- Open the file "*FPGA_Shield-cli.ino*" and press"**Upload**" in the Arduino IDE. Wait until the program finishes uploading.



- Now open **Tera Term** and connect to Arduino's **COMx**. (The correct Tera Term configurations are described in 4.3 .



  Pressing the **Enter key** on the keyboard will display the **Menu**, also known as the CLI (Command Line Interface).

The menu is divided into three main sections:

- **MENU Main**
  - Contains the main options: FPGA programming, reset, status, UART bridge, etc.
  - These are the most frequently used functions during development.
- **MENU SPI-NOR**
  - Options dedicated to communication with the SPI-NOR memory and debug functions.
- **Software Version**
  - Displays the current release of the firmware installed on Arduino.

All available options will be described in detail in later chapters.

Educational path of the chapter

In the following subsections we will create the first VHDL project, generate the bitstream file, and program the FPGA using two different methods. We will also see how to generate the binary file for the **SNOR Flash** memory, making the FPGA module independent of Arduino.

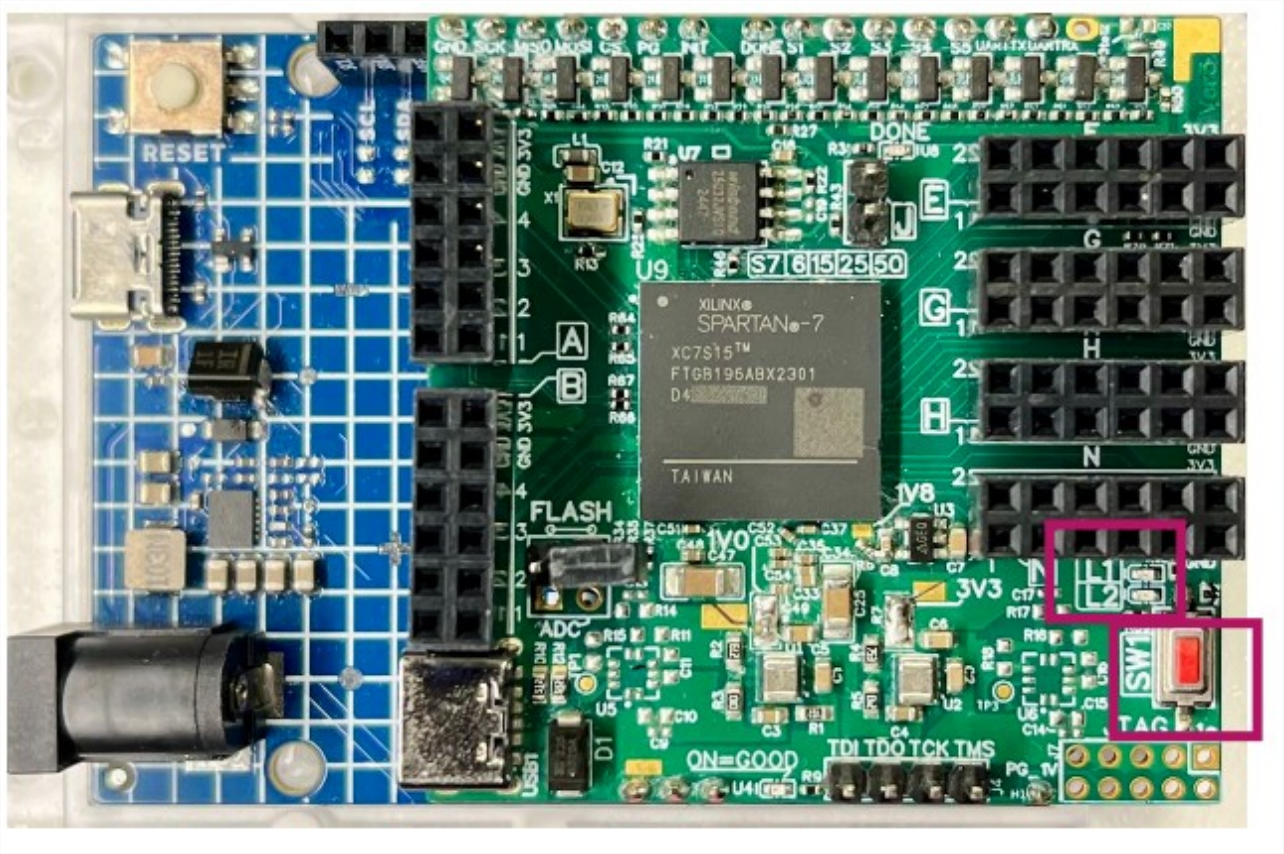
Here are the steps we will follow:

- First VHDL Project 2.1.
- Upload bitstream (method 1) 2.2 .
- Upload bitstream (method 2) 2.3 .
- Upload binary file in FLASH-SNOR (method 1) 2.4 .
- Upload binary file in FLASH-SNOR (method 2) 2.5 .

Before proceeding, it is necessary to have **Xilinx Vivado** installed on your PC. Please refer to chapter 4.4 for the installation guide.

2.1 First VHDL Project

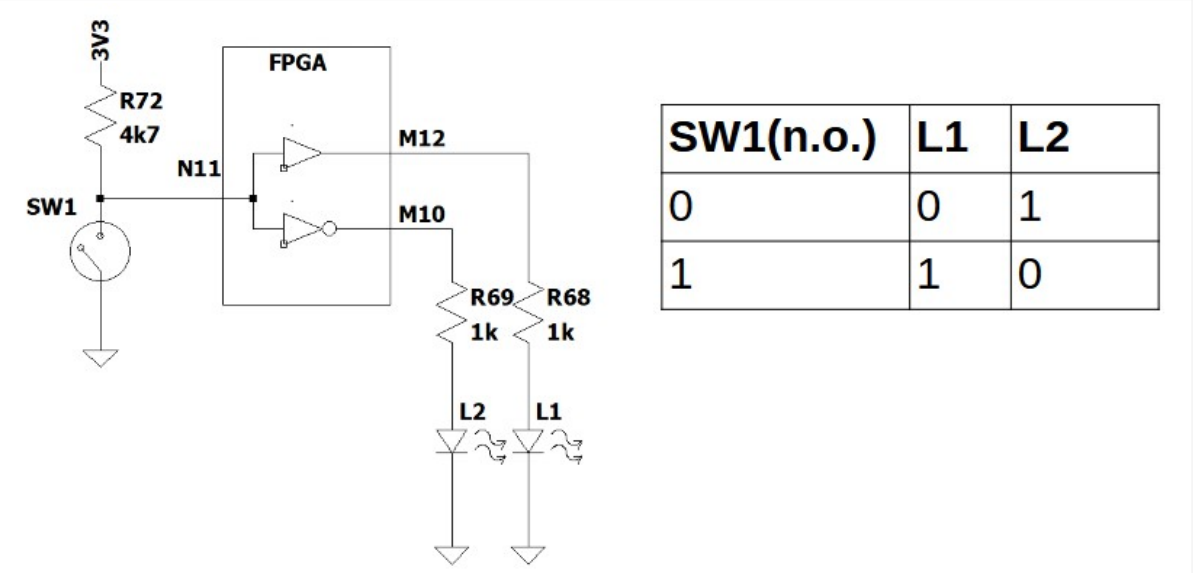
As a first VHDL exercise, we will use the two LEDs available on the "MODULO FPGA SPARTAN7" ***L1*** and ***L2*** and the push button ***SW1***.



We will control the two LEDs on the board with the button according to the following logic:

- When the **SW1** button is **not pressed**, the FPGA reads a logical value **“1”** thanks to the external (on-board) pull-up. In this state:
  -LED **L1** will be ON
  -LED **L2** will be OFF
- When the **SW1** button is **pressed**, the FPGA reads a logical value **“0”**. In this state:
  -LED **L2** will be ON
  -LED **L1** will be OFF

Schematic principle:

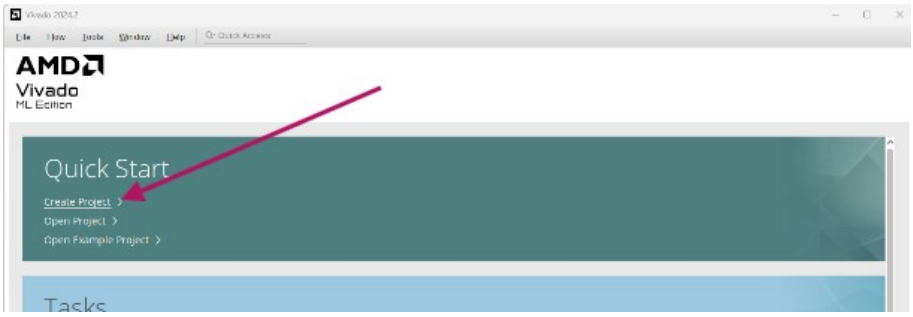


SW1(n.o.)	L1	L2
0	0	1
1	1	0

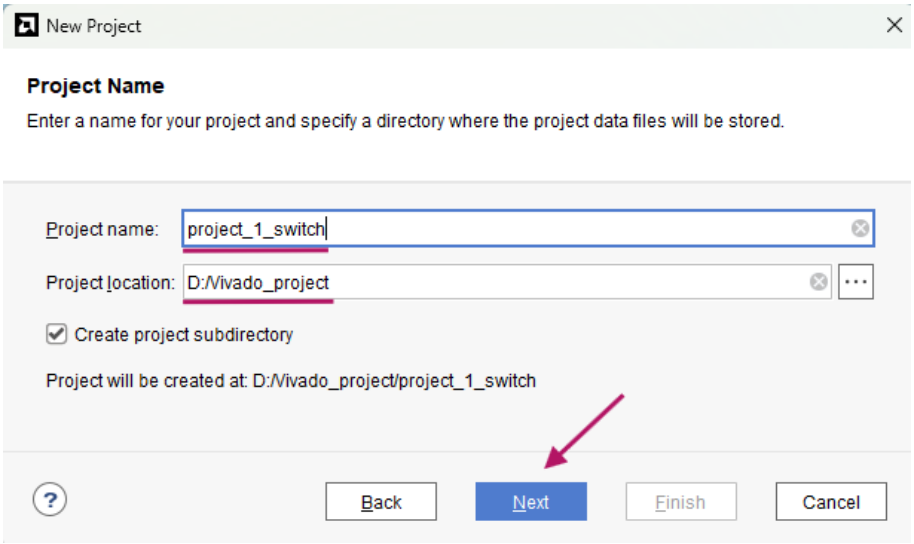
Once the behavior is defined, we need to create a VHDL project in Vivado.

Creating the project in Vivado

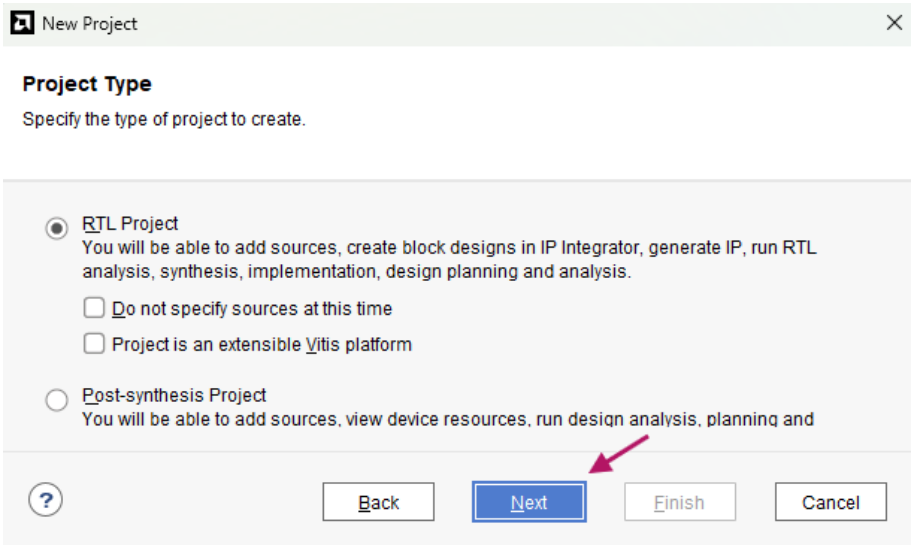
- Open Vivado and select **Create Project** :



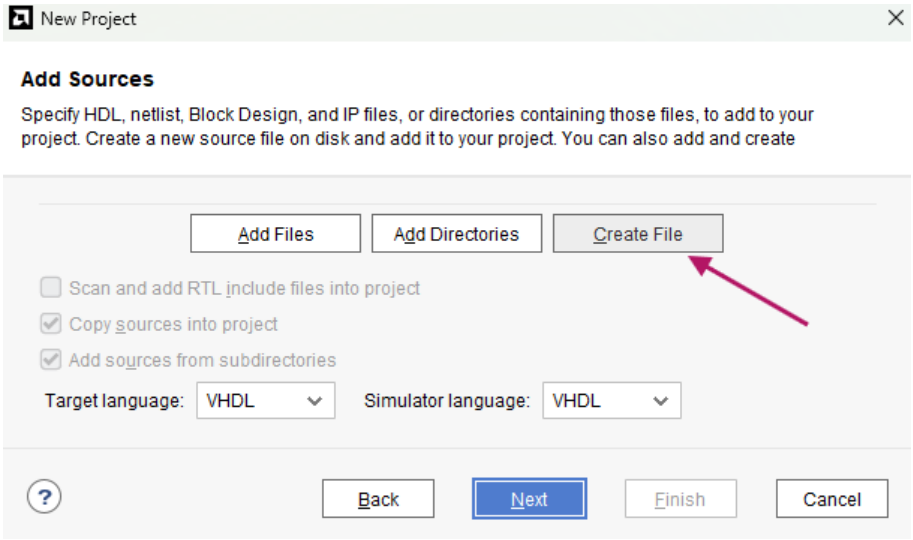
- Choose the project path



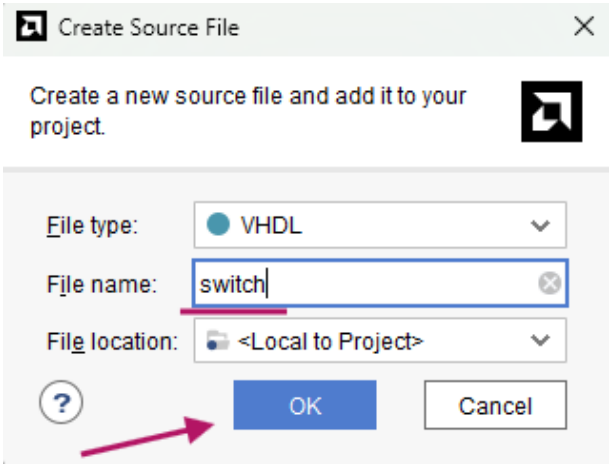
- Select **RTL Project:**



- Create the file where we will copy our VHDL code

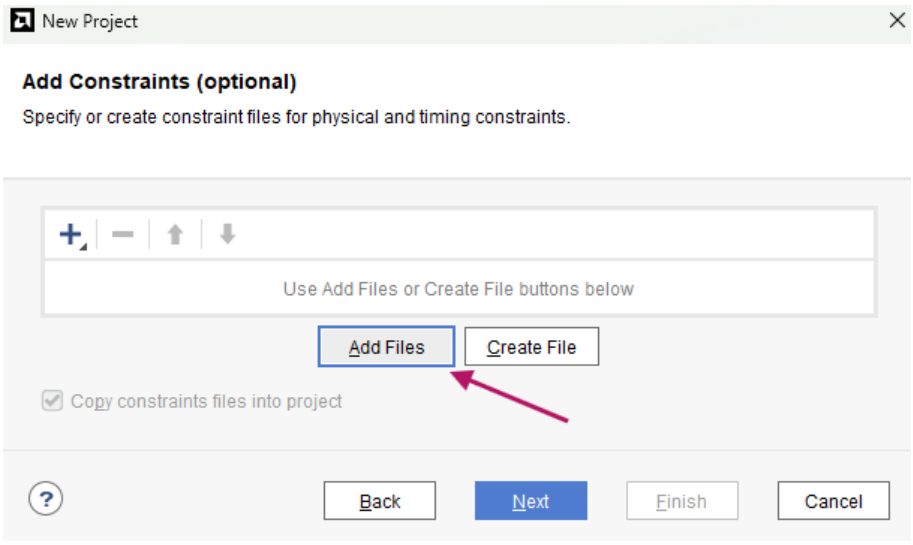


- Choose the name of the VHDL file

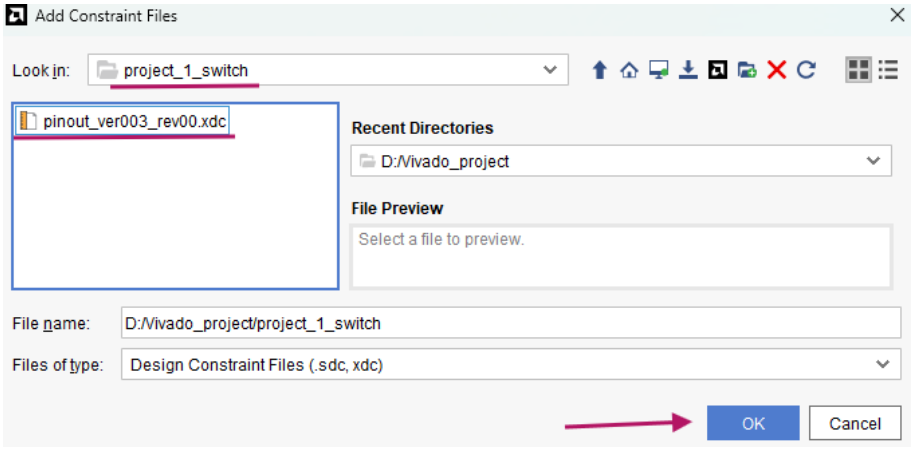


Physical pin assignment (file .xdc)

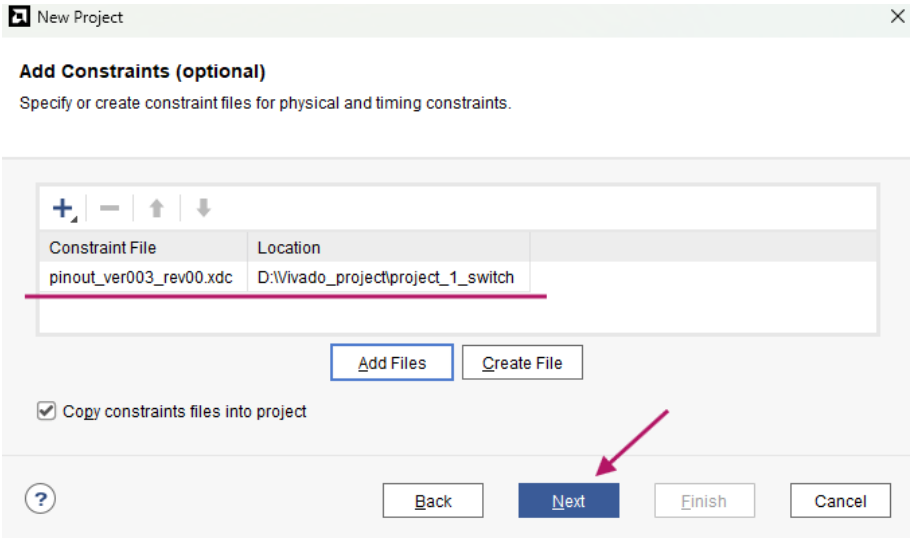
- Add the FPGA pinout file with the **.xdc** extension



- Download the .xdc file from the repository and copy it into the project folder:
  File .xdc: *https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/pinout/pinout_ver003_rev00.xdc*
  Example folder:
  D:\Vivado_project\project_1_switch
- Once copied, the .xdc file will be visible in Vivado → select it and press **OK**

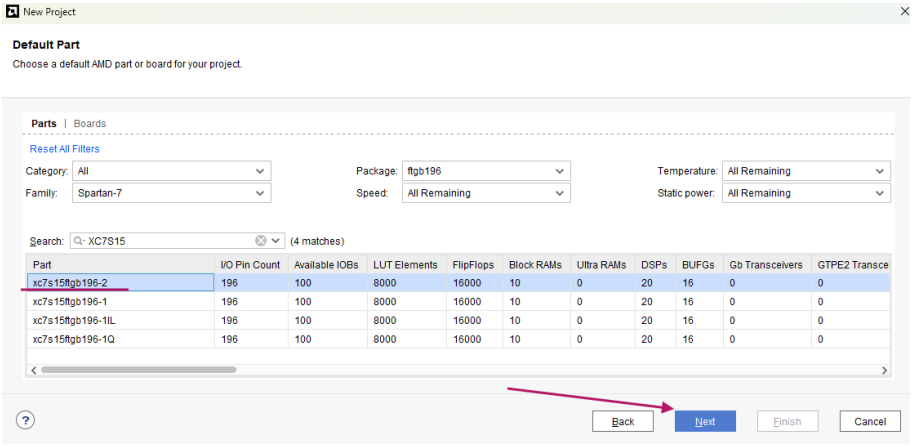


- The .xdc file is now added to the project

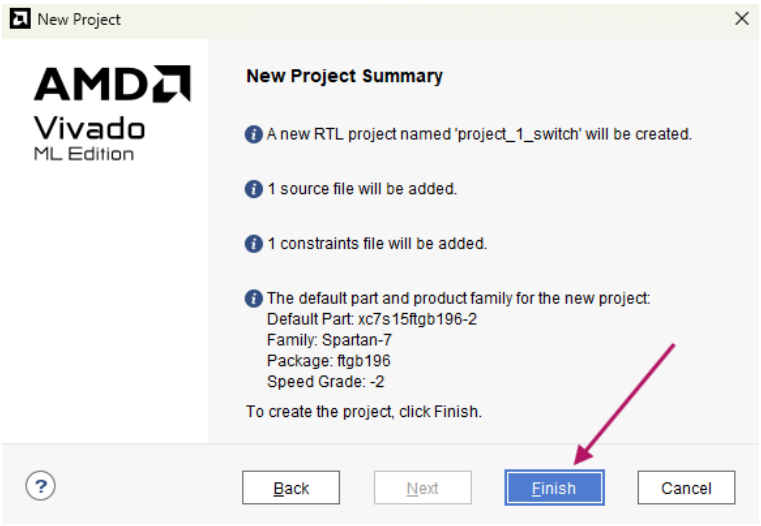


- Now select the **part-number** of the FPGA

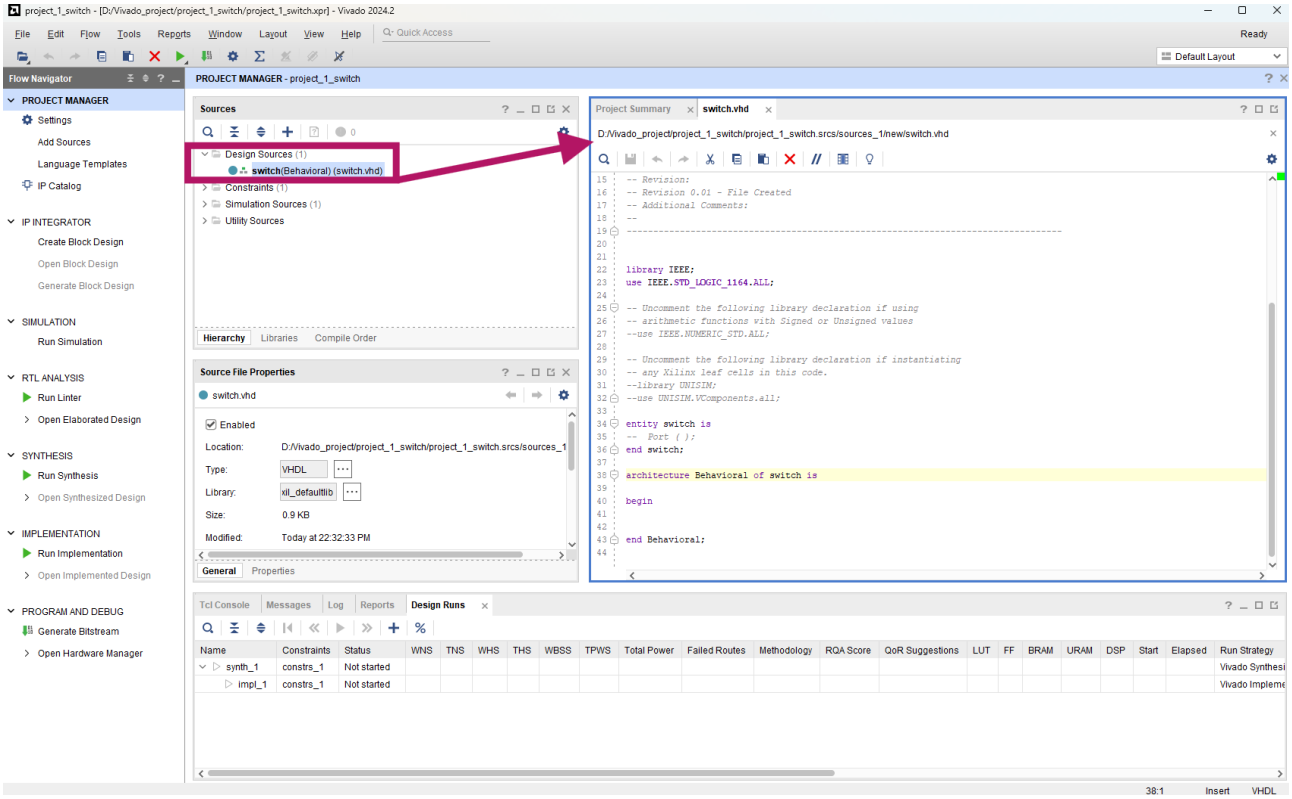
  ⚠ Make sure to refer to your specific board version (see 4.2)



- Press **Finish** → the project will be created



- Open the project. By double-clicking the *switch.vhd* file, the default VHDL template will open, ready for editing.

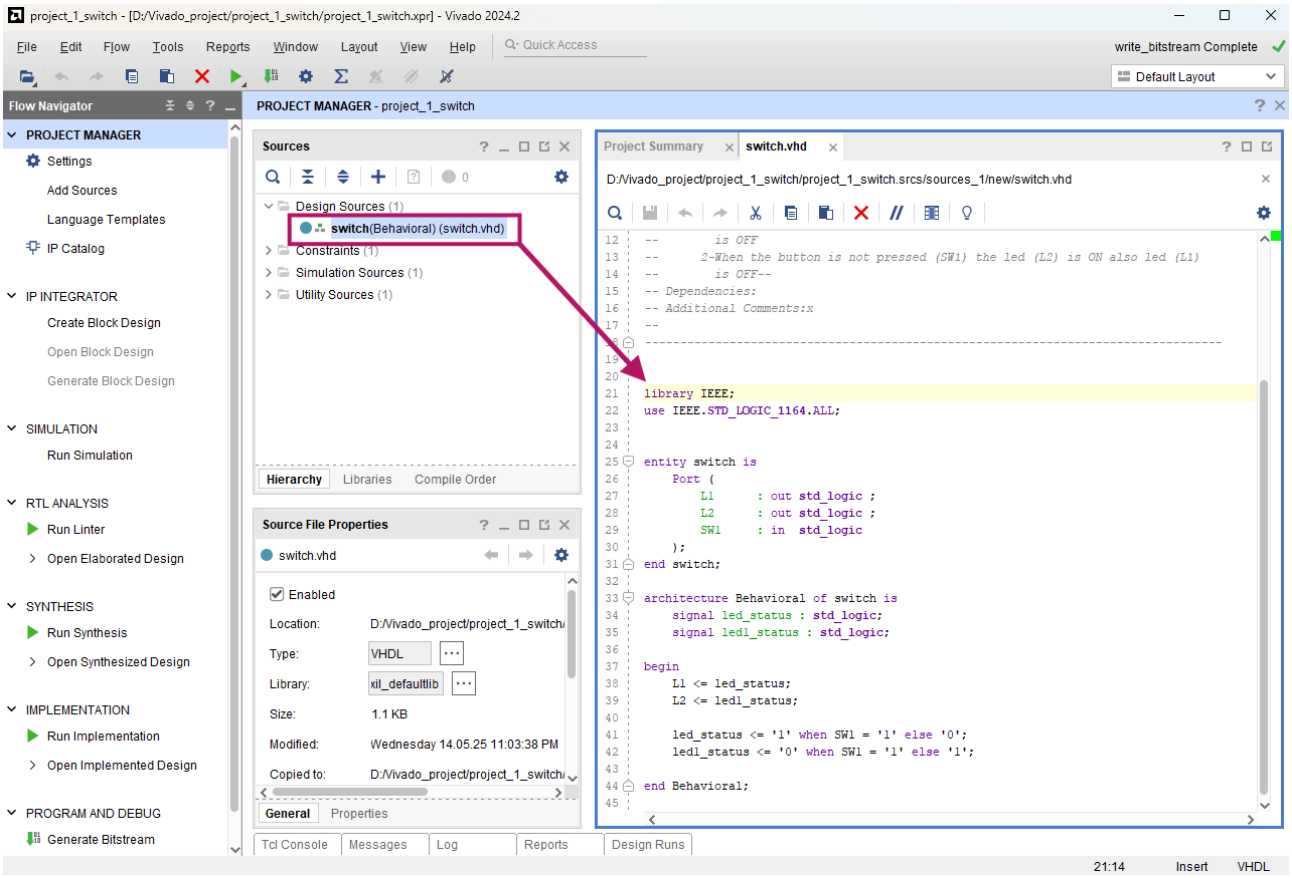


Example code

- This is the *switch.vhd* file describing the LED behavior:
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity switch is
    Port (
        L1      : out std_logic ;
        L2      : out std_logic ;
        SW1     : in  std_logic
    );
end switch;

architecture Behavioral of switch is
    signal led_status : std_logic;
    signal led1_status : std_logic;

begin
    L1 <= led_status;
    L2 <= led1_status;

    led_status <= '1' when SW1 = '1' else '0';
    led1_status <= '0' when SW1 = '1' else '1';

end Behavioral;
```

- Copy/import the file into your Vivado project and open it



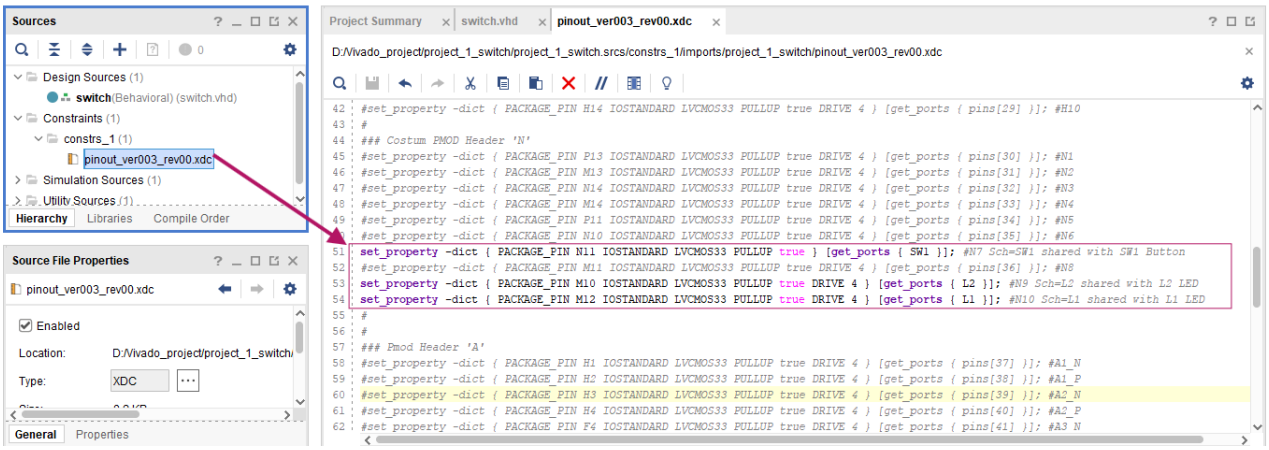
Pin assignment update

- We must now associate the **outputs** and **inputs** of the VHDL code to the physical pins of the FPGA.
  To do this, we edit the **.xdc file**, already prepared in the repository:
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/pinout_ver003_rev00.xdc

  These are the lines to uncomment:

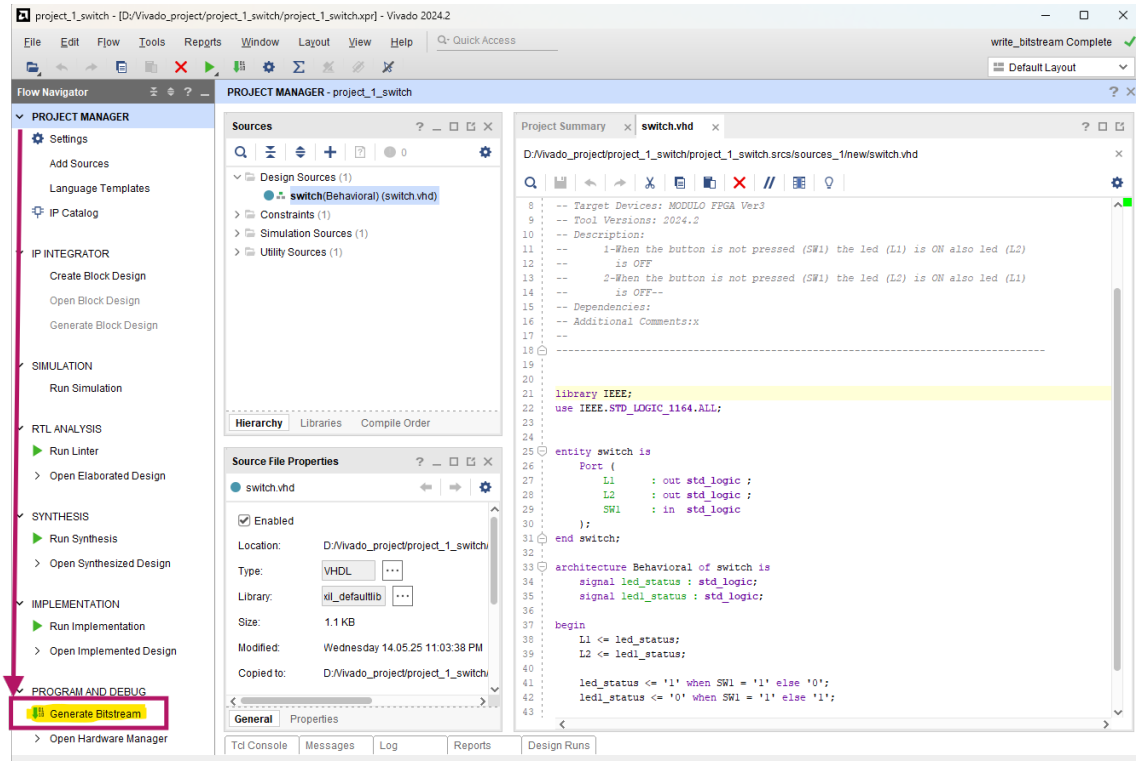
```
set_property -dict { PACKAGE_PIN N11 IOSTANDARD LVCMOS33 PULLUP
true } [get_ports { SW1 }]; #N7 Sch=SW1 shared with SW1 Button

set_property -dict { PACKAGE_PIN M10 IOSTANDARD LVCMOS33 PULLUP
true DRIVE 4 } [get_ports { L2 }]; #N9 Sch=L2 shared with L2 LED

set_property -dict { PACKAGE_PIN M12 IOSTANDARD LVCMOS33 PULLUP
true DRIVE 4 } [get_ports { L1 }]; #N10 Sch=L1 shared with L1 LED
```

  It will look like this:

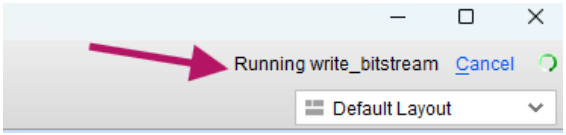


Generate the bitstream

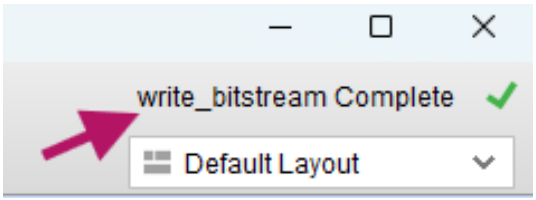
- At this stage, we can generate the **bitstream file**, which will contain your first VHDL code
- Click **Generate Bitstream**



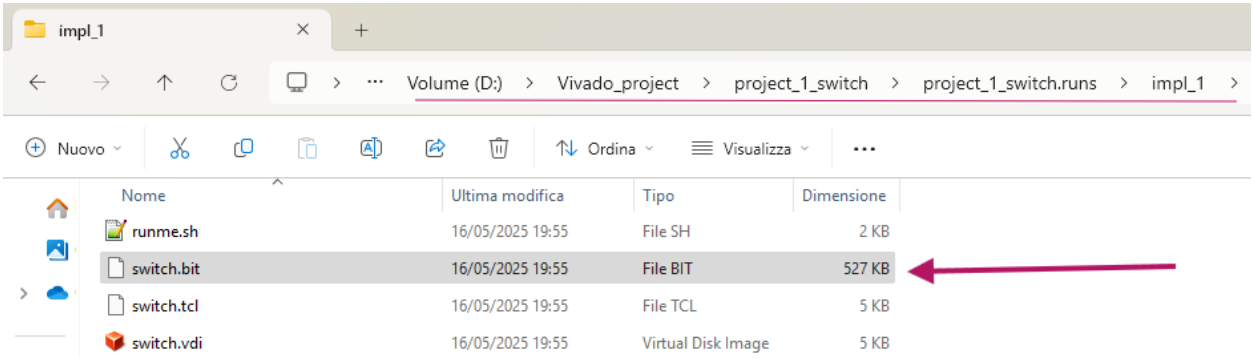
- The progress will appear in the upper-right corner of Vivado



- If successful, the status will show **Complete**



- The generated bitstream file will be located in the project folder, e.g.: *D:\Vivado_project\project_1_switch\project_1_switch.runs\impl_1*

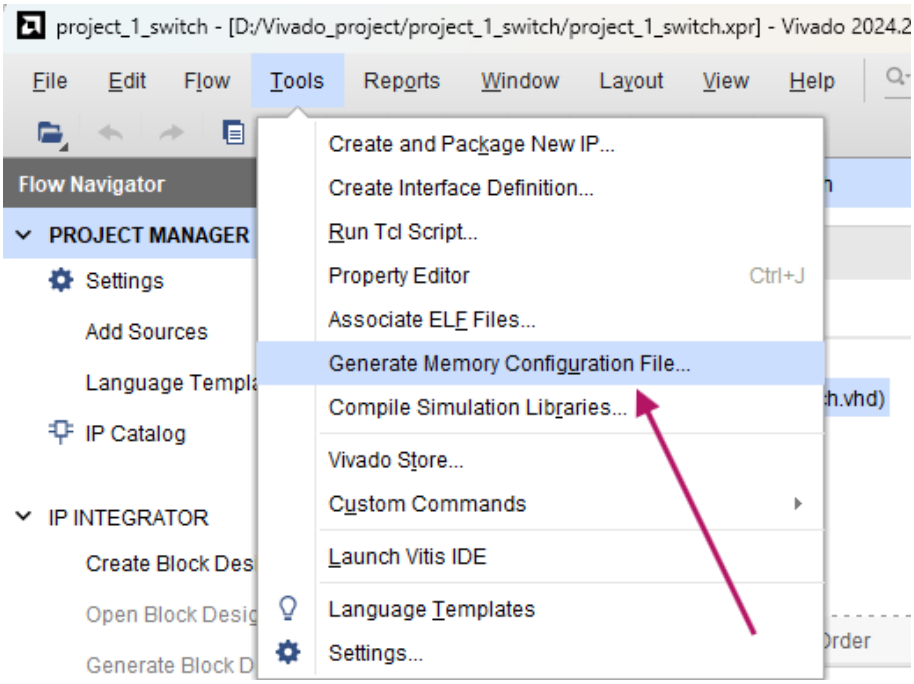


The bitstream file (*.bit*) will be used to configure the FPGA every time it is reset or powered on.

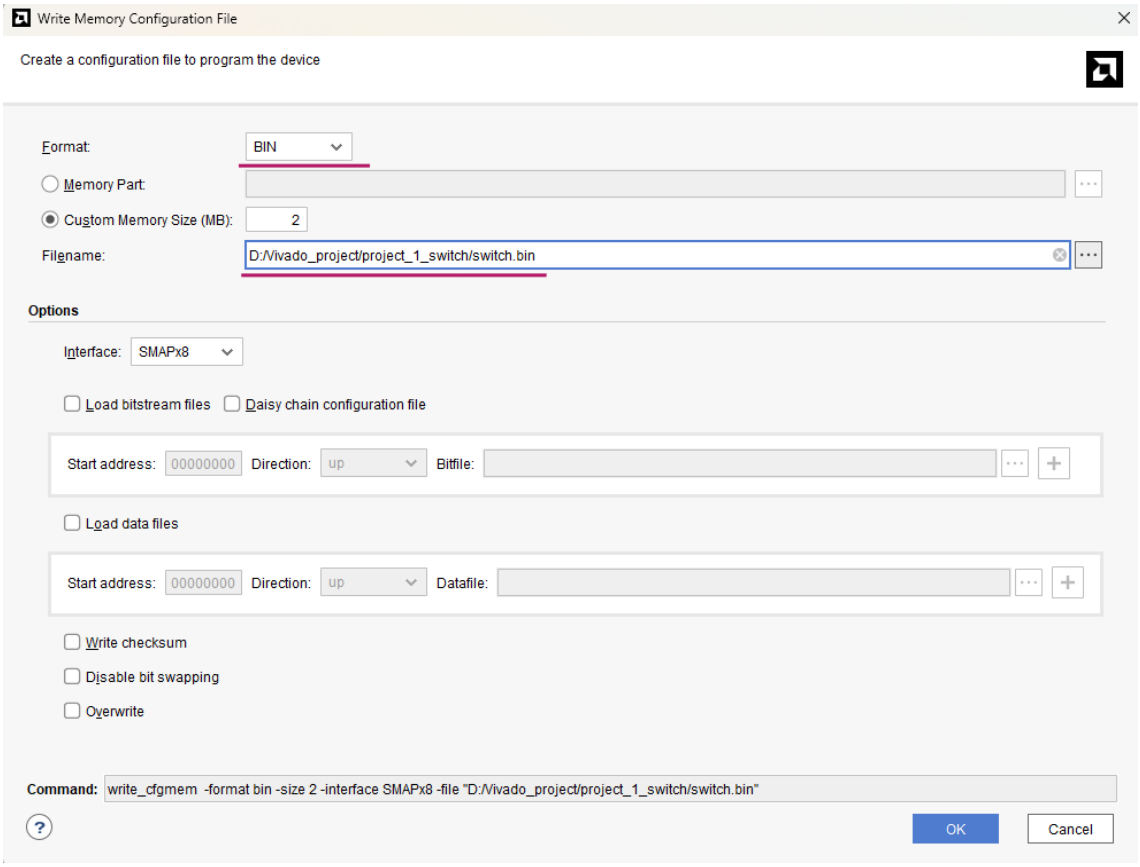
However, to make the "MODULO FPGA SPARTAN7"self-configuring at each restart, we must generate a **binary file (.bin)** for the SNOR memory.

Create the binary file

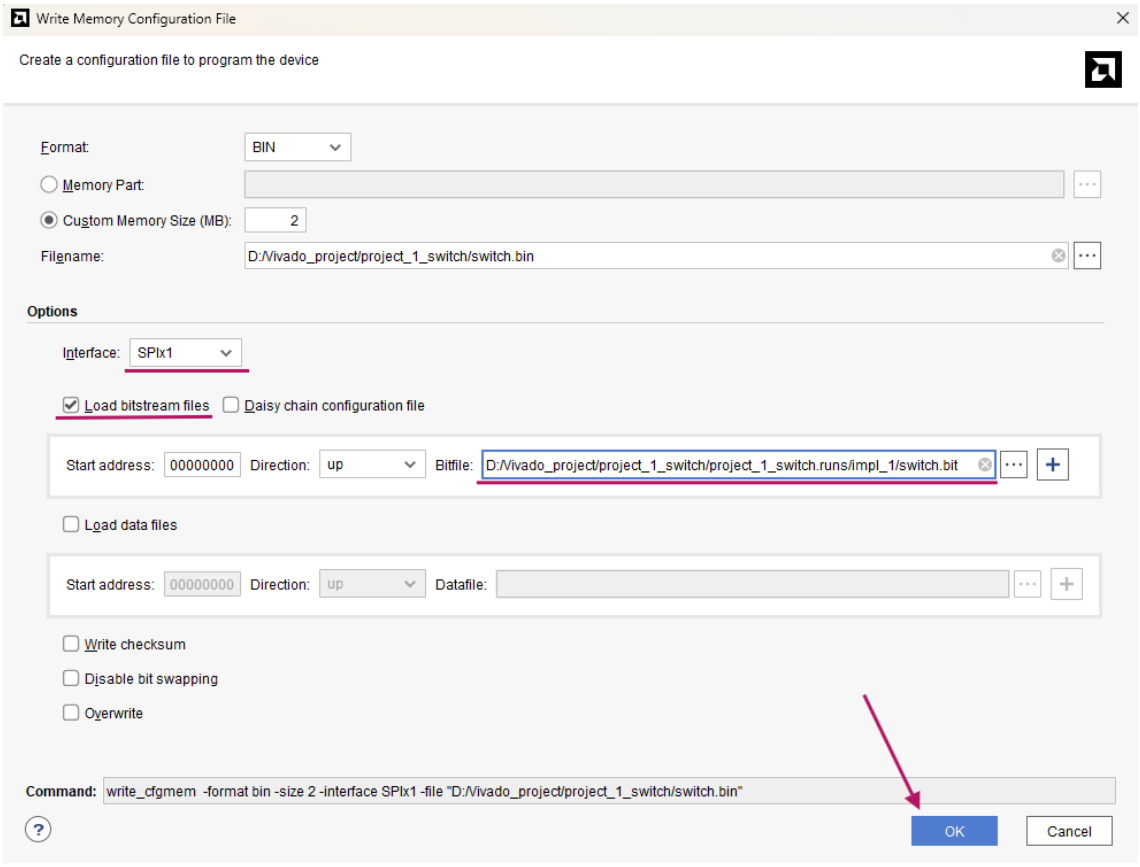
- In Vivado, go to **Tools →**



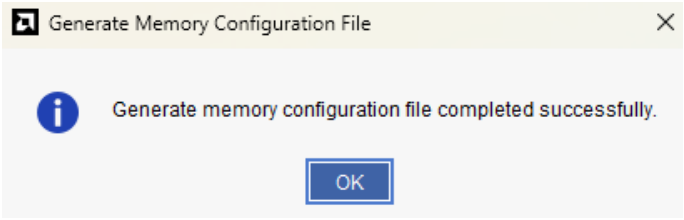
- Select format **BIN**, size **2MB**, and choose the output filename



- Select **SPIx1** and the bitstream *switch.bit* from the project folder



- The binary file will now be generated



2.2 Upload bitstream (method 1)

In this chapter we will upload the bitstream file using Arduino R4 and Tera Term.

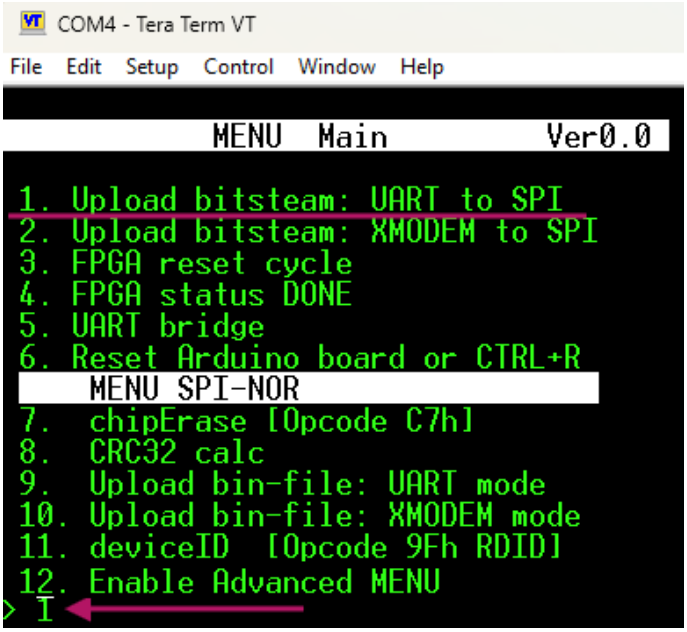
The file on the PC will be sent to Arduino via serial connection, and Arduino will converter the UART serial data into SPI data.

In this example, we will use Tera Term because it satisfied two foundamental requirementes:

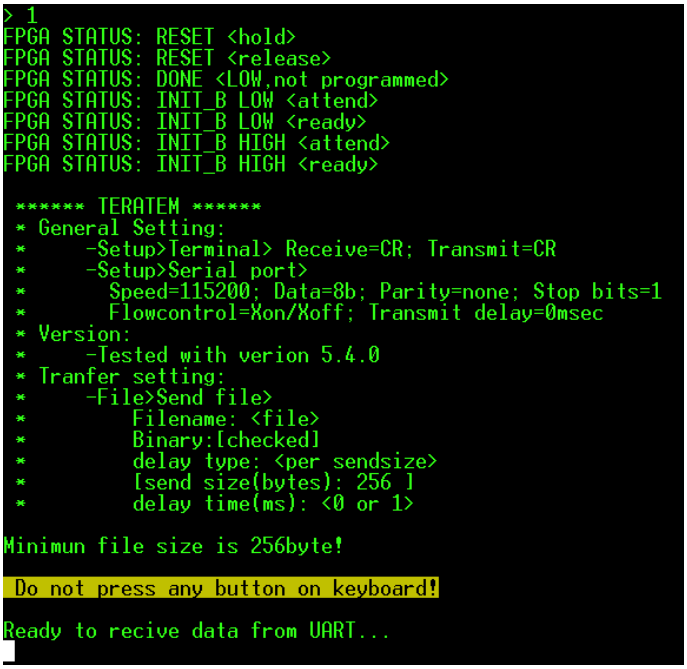
1- Support for *XON/XOFF Flow Control.*

2- Transmission of a maximum of *256 bytes at a time.*

- Select option **1** from the **Menu** and press **Enter** :

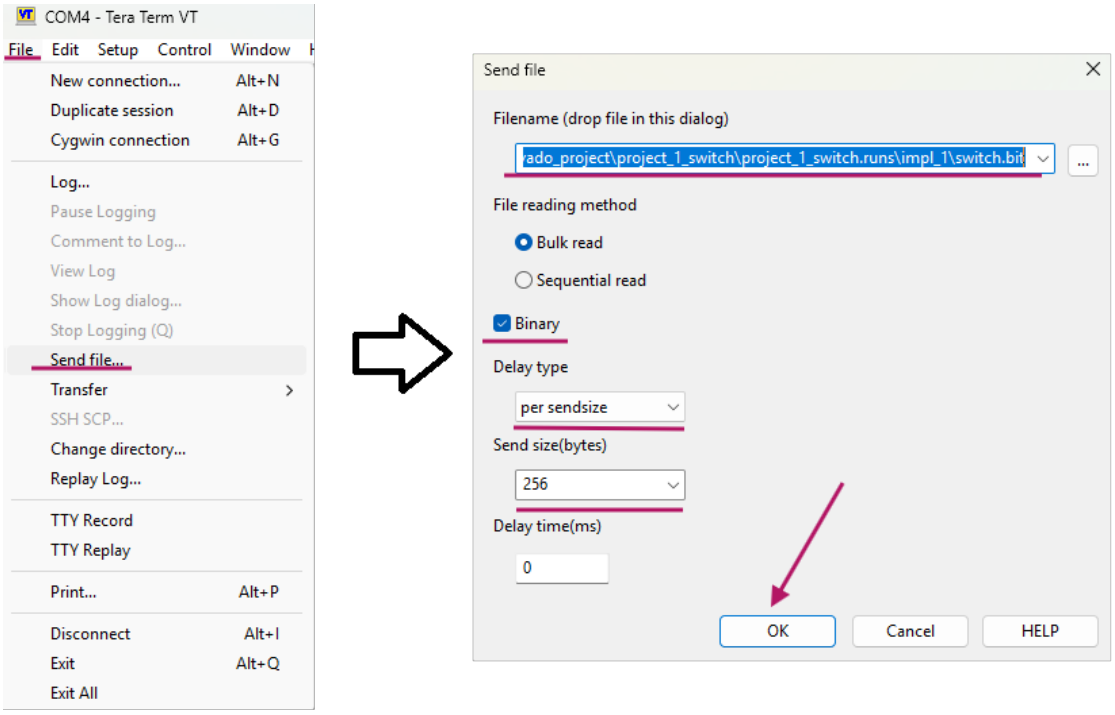


- Arduino will wait for the bitstream file to be sent m:

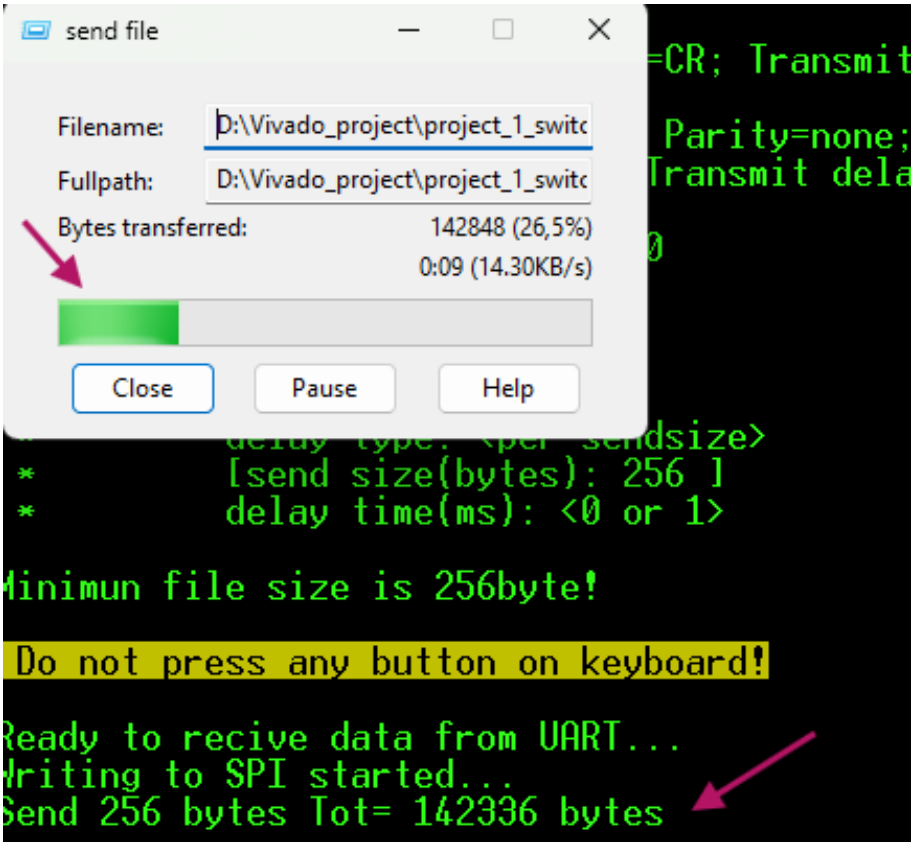


- In Tera Term, go to **File → Send file…** and select the bitstream file from:
  D:\Vivado_project\project_1_switch\project_1_switch.runs\impl_1\switch.bit
  or form GitHub:
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bit

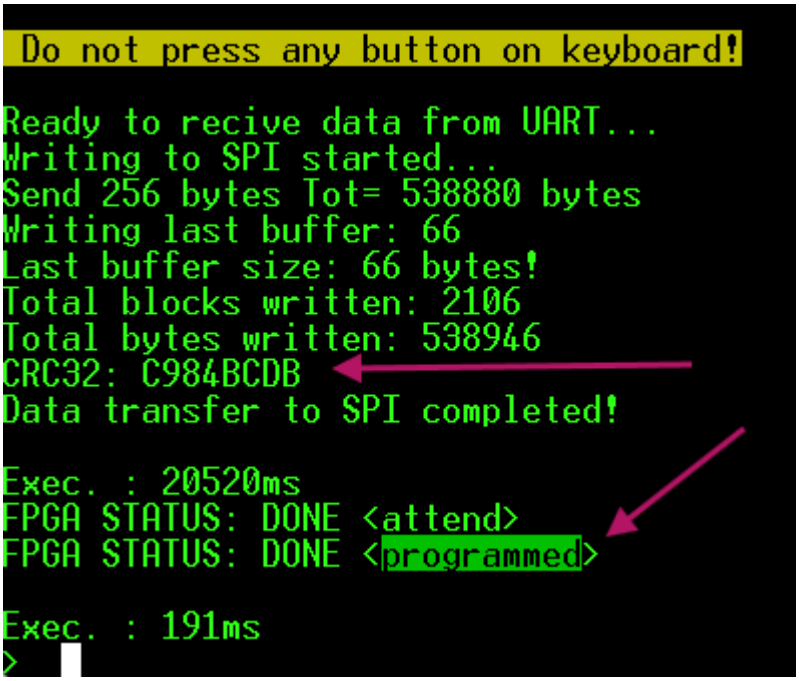
  Check **Binary**
  Select **Delay type / per sendsize = 256**
  Press **OK**



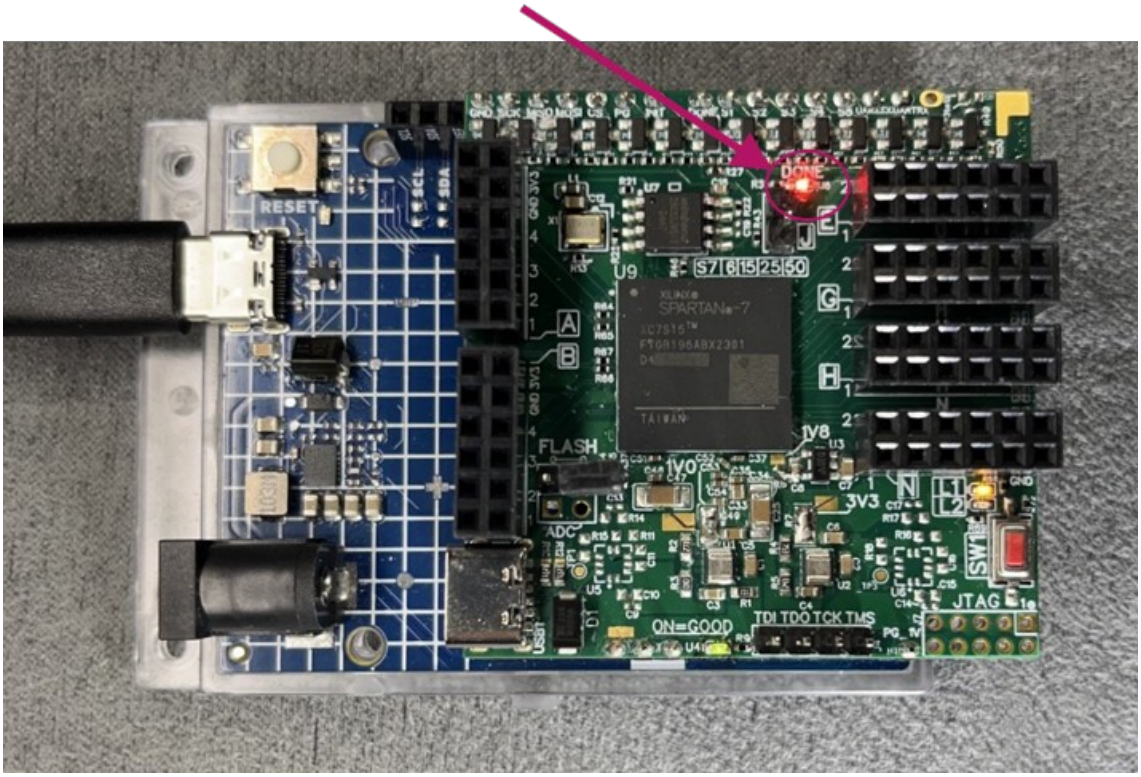
- After pressing OK, the transmission will start



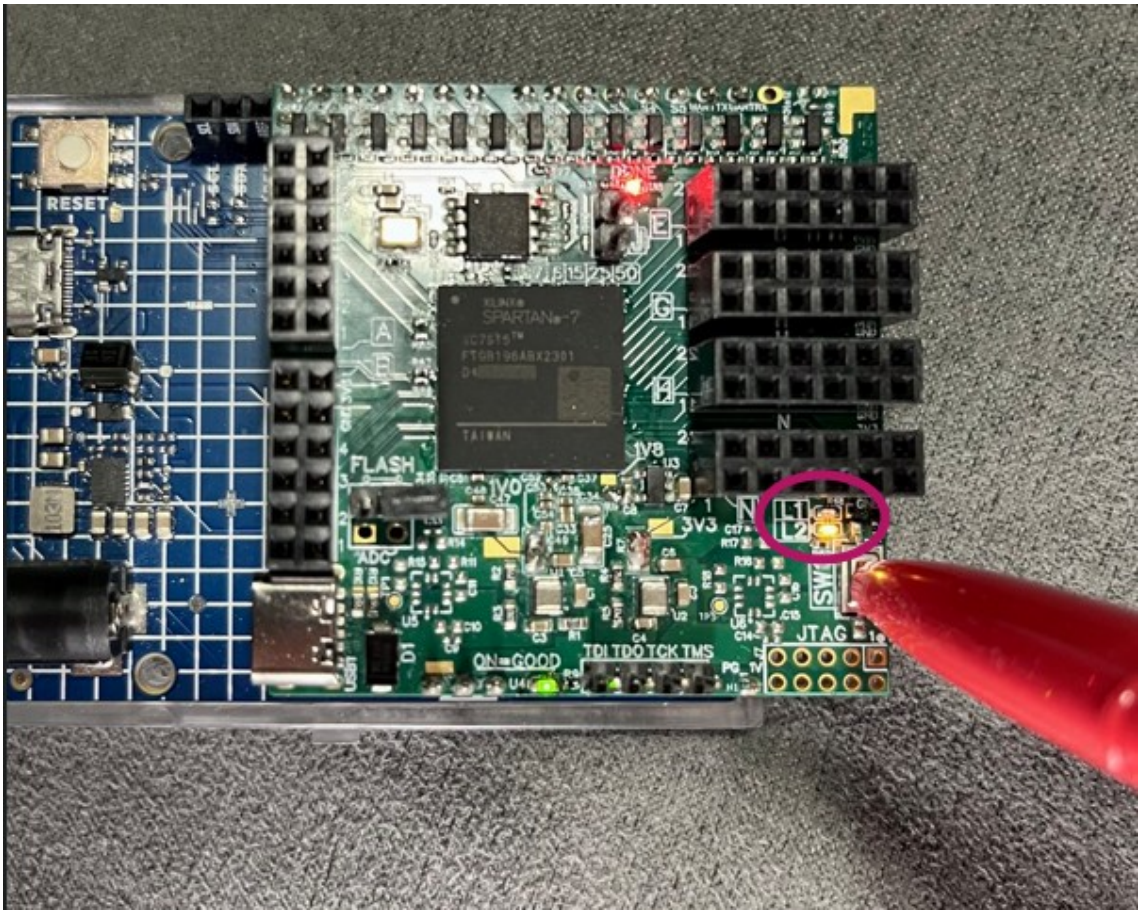
- At the end of the transmission, verify that the FPGA has been programmed by checking the output message **“programmed”** .



- Look at your board… the FPGA is loaded! **“ON FIRE”** FIRE"



- LED **L1** is ON while LED **L2** is OFF, as per the project .
- Now press the **SW1 button** → LED **L1** turns OFF while LED **L2** turns ON, as expected



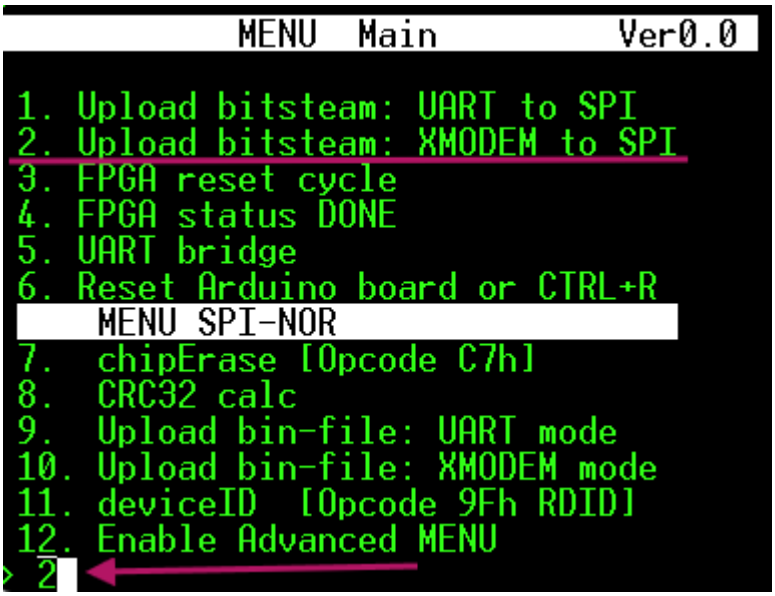
END

2.3 Upload bitstream (method 2)

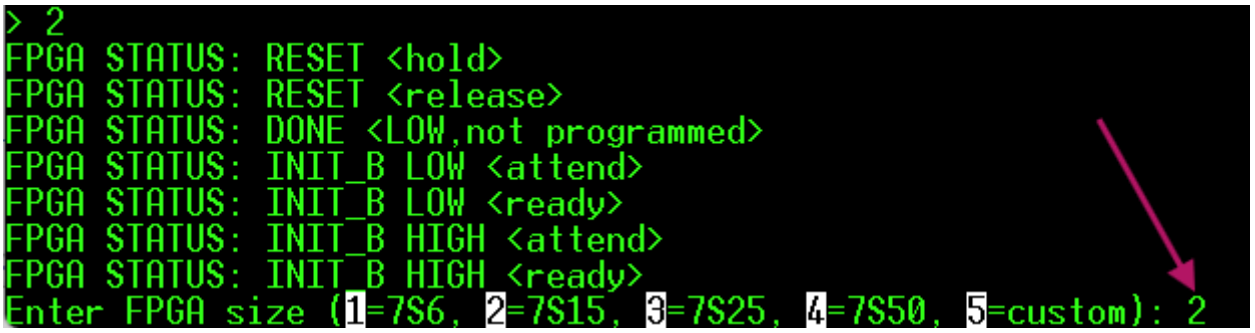
In this method we will again use **Arduino R4** and **Tera Term**, but this time we will rely on the **XMODEM protocol**.
The file on the PC will be sent to Arduino via serial connection, and Arduino will convert the UART serial data into SPI data.
In this example we use **Tera Term** because it provides XMODEM transfer support.

Unlike chapter 2.2 here it is only necessary to specify the file size to be transferred.

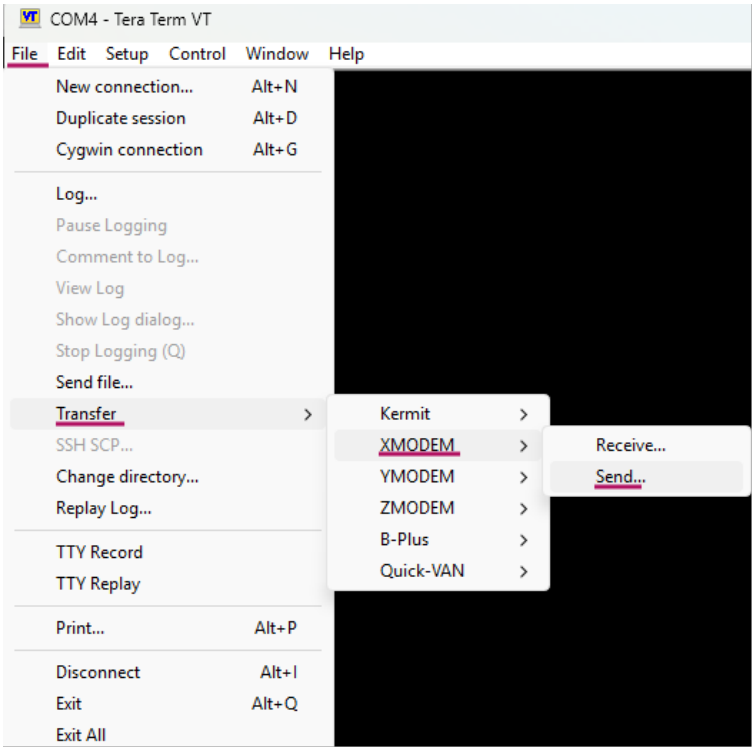
- Select option **2** from the **Menu** and press **Enter:**



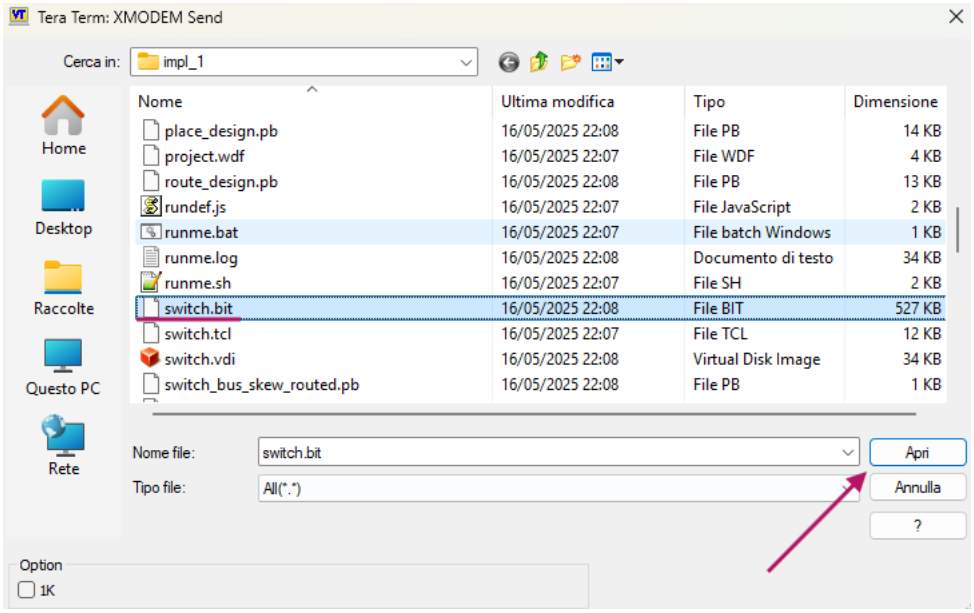
- Select the FPGA mounted on the board (**S715**) by typing **2** and then pressing **Enter** :



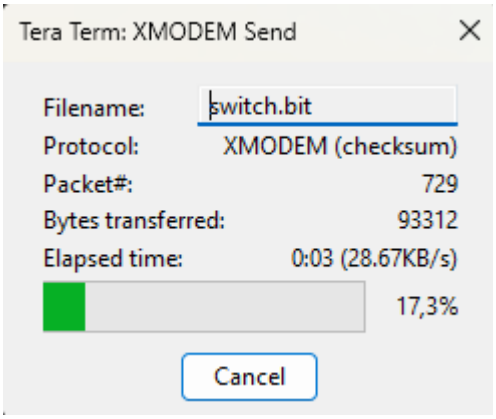
- In Tera Term go to **File → Transfer → XMODEM → Send…** and select the bitstream file:
  D:\Vivado_project\project_1_switch\project_1_switch.runs\impl_1\switch.bit
  or GitHub
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bit



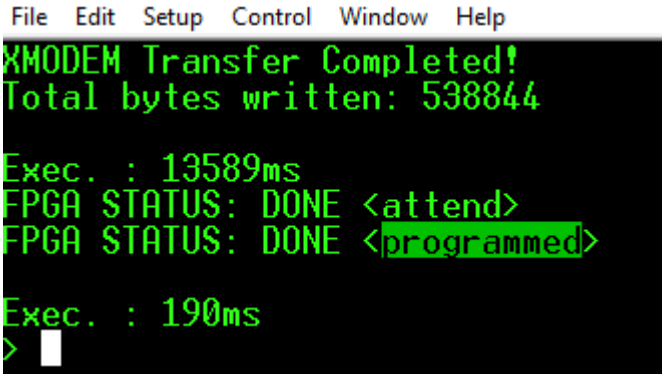
- Select the file *switch.bit*



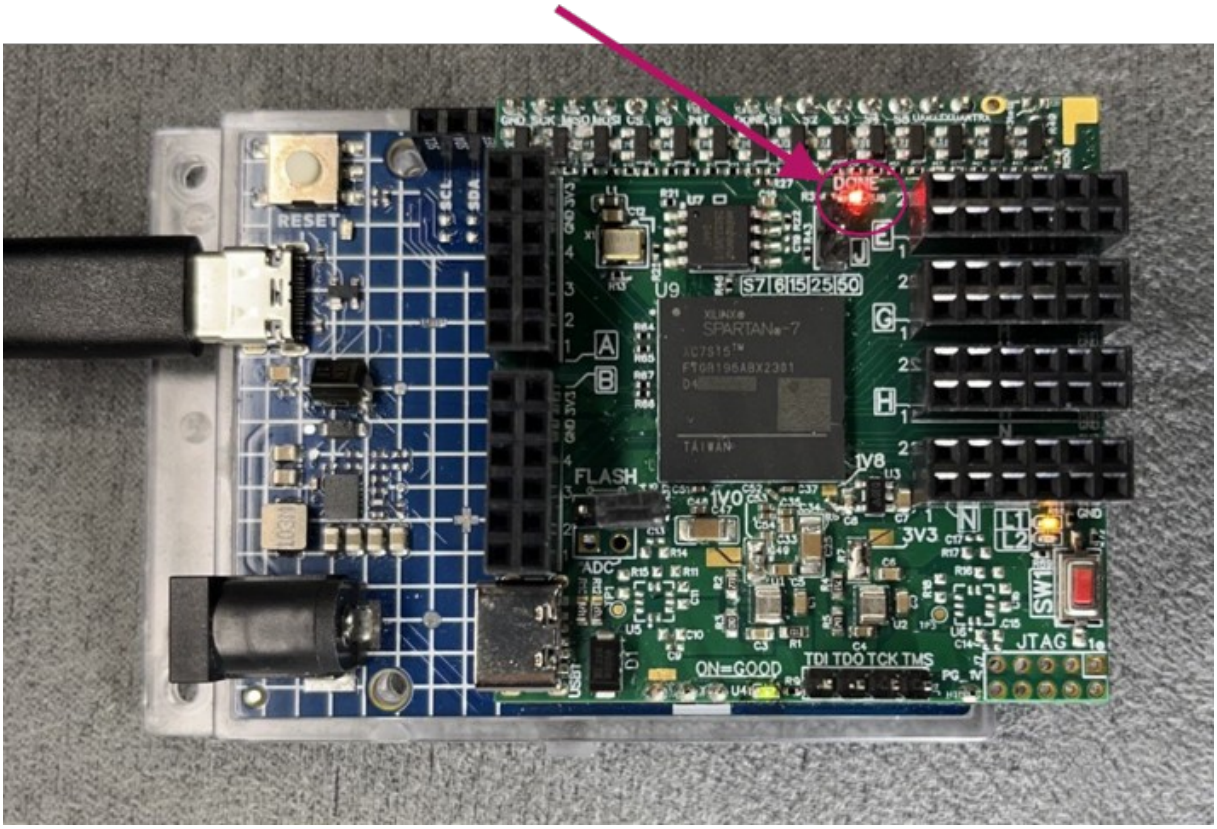
- A window will open showing the file upload progress bar:



- Once completed, the FPGA will be loaded



- Look at your board… the FPGA is programmed! **“ON FIRE”**

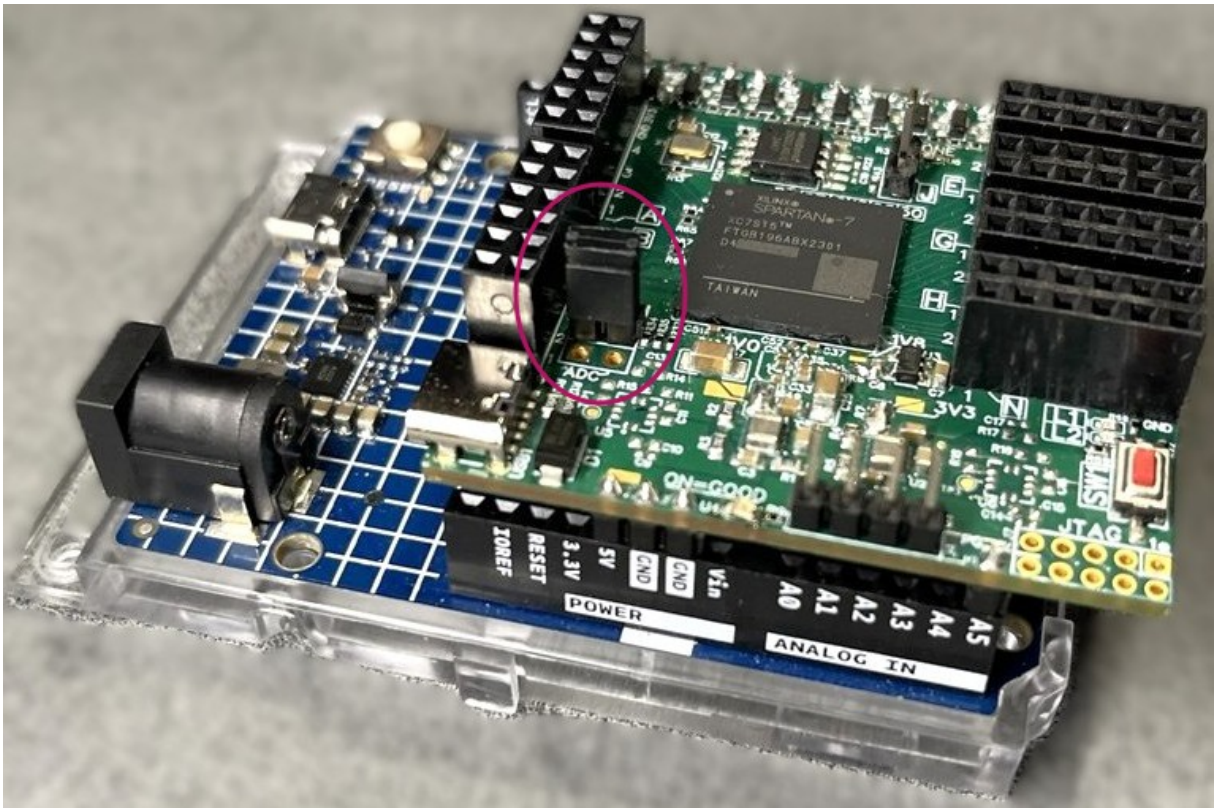


2.4 Upload binary FLASH SNOR (method 1)

In this chapter we will upload the **binary file** into the **FLASH-SNOR** of the "MODULO FPGA SPARTAN7" using **Arduino R4** and **Tera Term**.

⚠ It is important to **erase the FLASH-SNOR** before every new binary upload.

- Close the **FLASH jumper**



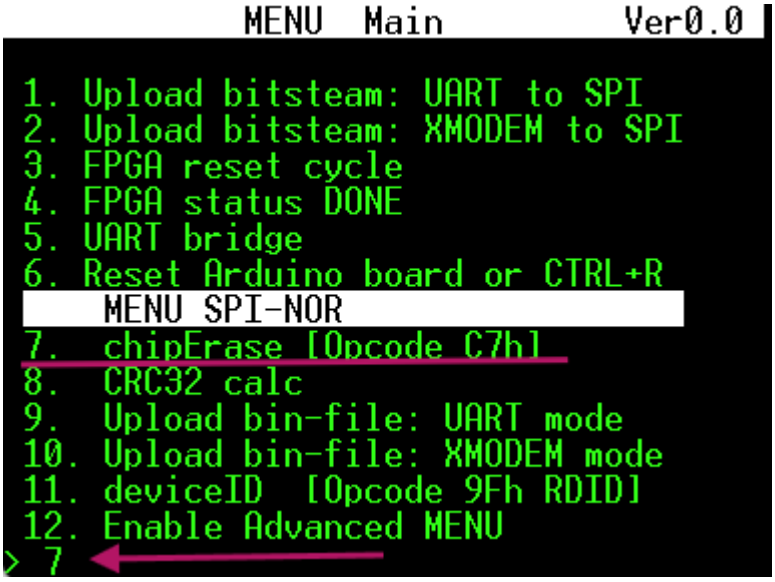
➢ By closing the FLASH jumper, the FPGA enters stand-alone mode: at power-on or reset, it will automatically load the binary from the SNOR.
In this case, we no longer need to reload the FPGA every time, and the "MODULO FPGA SPARTAN7 can be used stand-alone, powered through its USB-C port.

The file on the PC will be sent to Arduino via serial connection, and Arduino will convert the UART serial data into SPI data.
In this example we use **Tera Term**, since it satisfies two key requirements:

1- Support for **XON/XOFF Flow Control**

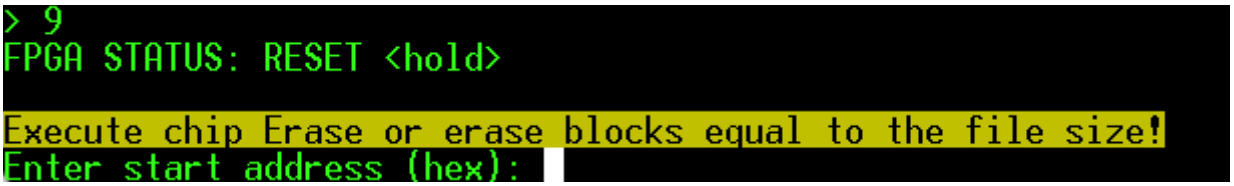
2- Transmission of a maximum of **256 bytes at a time**

- Erase the FLASH-SNOR, then press **Y** and hit **Enter** :



- The FLASH is now empty

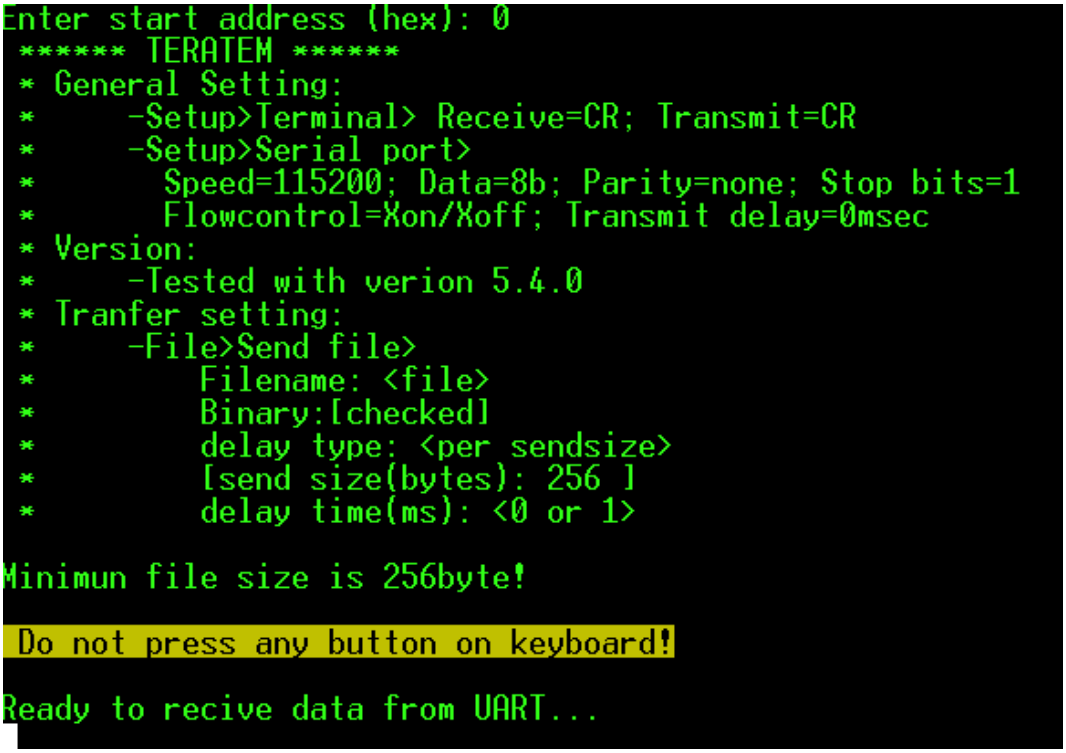
- Now we can load the binary file into the FLASH-SNOR → press **9** and hit **Enter**



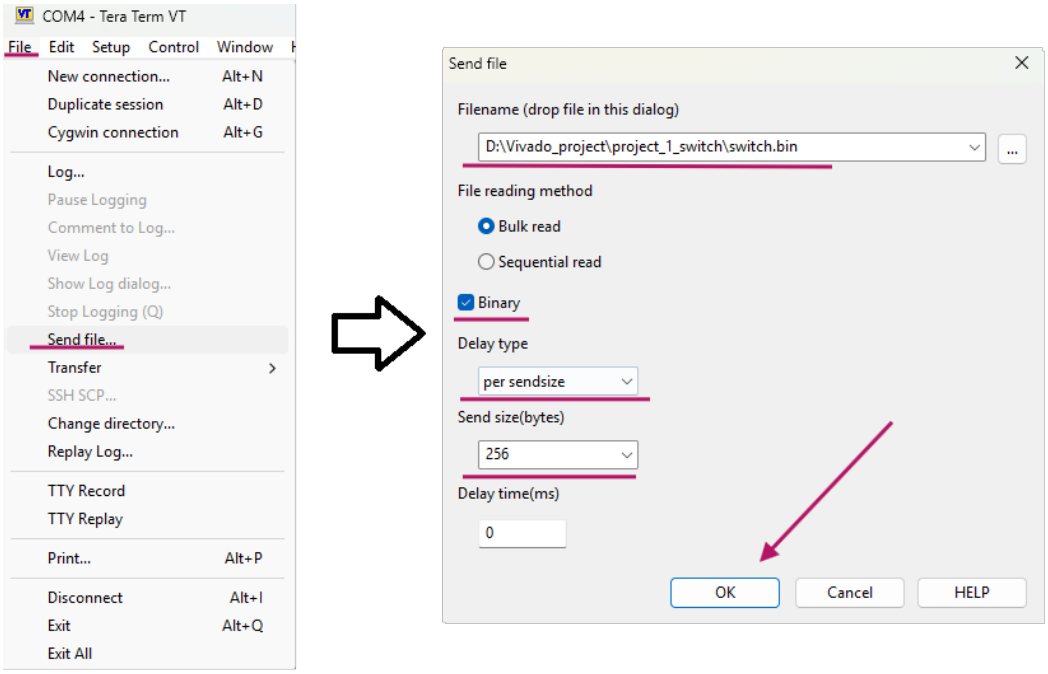
At this stage, a reminder will appear to erase the FLASH before writing, and then you will be asked to select the start address.

We will start erase from **0**.

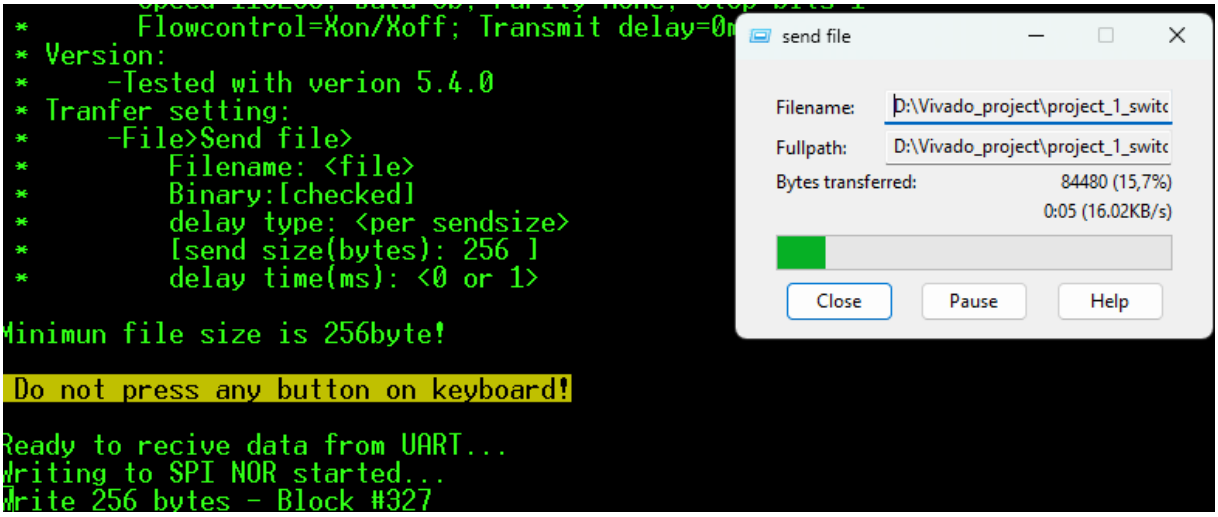
- Arduino is now ready to receive the binary file, also available in the repository: *https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bin*



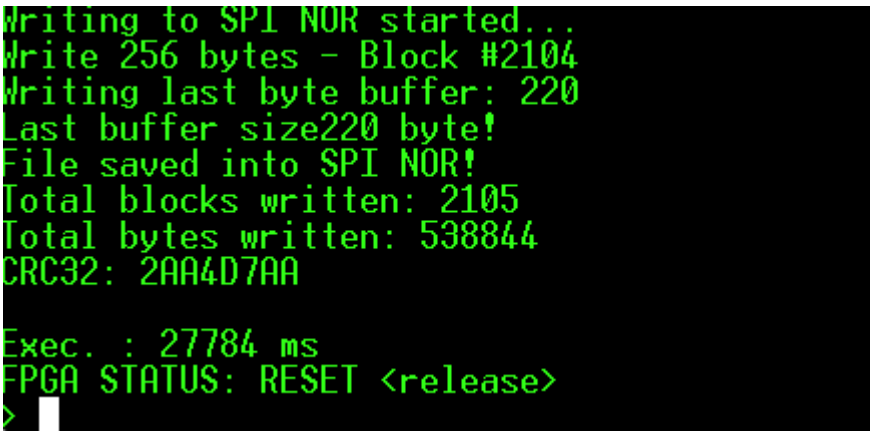
- In Tera Term go to **File → Send file…** and select the binary file
  Check **Binary**
  Select **Delay type / per sendsize = 256**
  Press **OK**



- The transfer will start



- Once the binary has been written into the **FLASH-SNOR**, Arduino automatically releases the FPGA reset.
  Since the FPGA is in **MASTER MODE**, it will automatically load the binary file.
  The FPGA is programmed! **"ON FIRE"**



  Alternatively, to check the FPGA status, select option **4** from the menu.

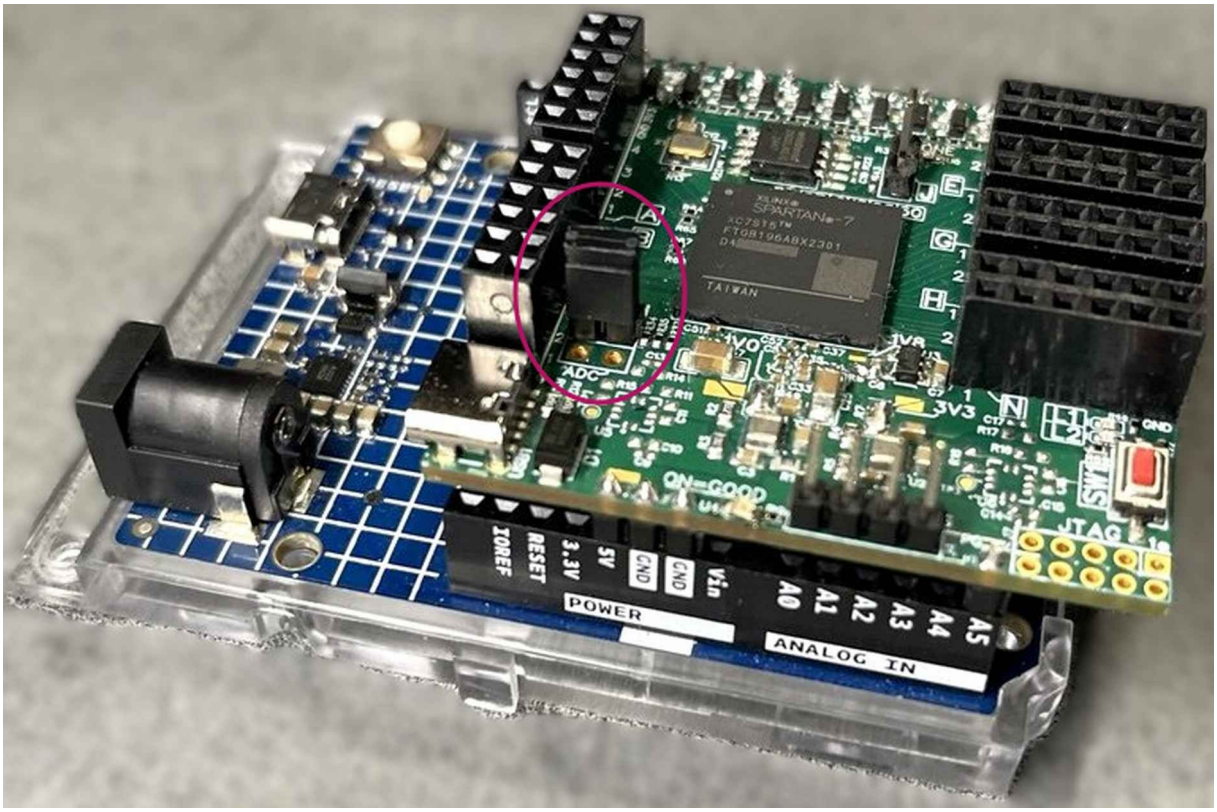
➢ **Tip:**
Disconnect power and unplug the FPGA Module from Arduino.
Then power the FPGA Module using its own USB-C port… you will notice that Arduino is no longer required.

2.5 Upload binary FLASH SNOR (method 2)

In this chapter we will upload the **binary file** into the **FLASH-SNOR** of the **"MODULO FPGA SPARTAN7"** using **Arduino R4** and **Tera Term**.

⚠ It is important to **erase the FLASH-SNOR** before every new binary upload.

- Close the **FLASH jumper**

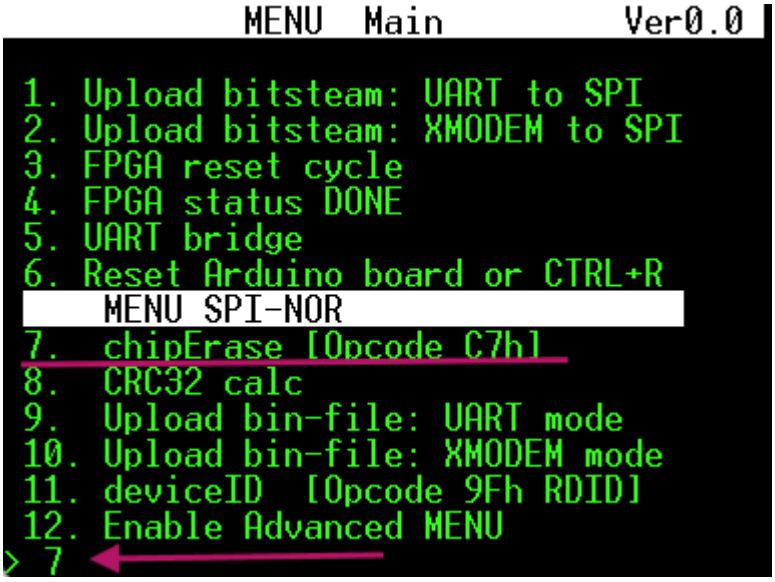


By closing the FLASH jumper, the FPGA enters **stand-alone mode**: at power-on or reset, it will automatically load the binary from the SNOR.
In this case, we no longer need to reload the FPGA every time, and the **"MODULO FPGA SPARTAN7"** can be used stand-alone, powered through its USB-C port.

The file on the PC will be sent to Arduino via serial connection, and Arduino will convert the UART serial data into SPI data.
In this example we use **Tera Term**, since it provides **XMODEM transfer** support.

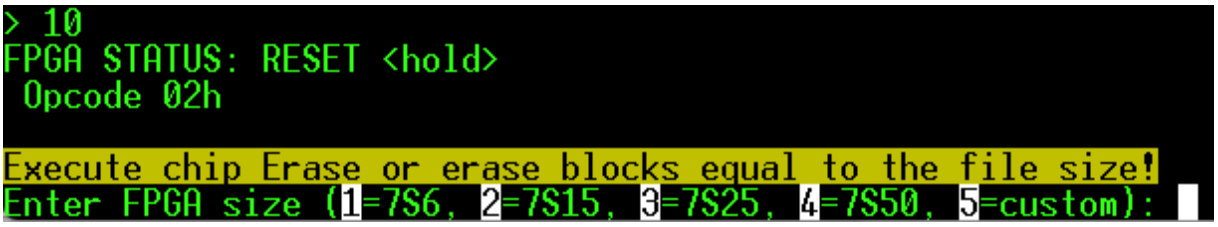
Unlike chapter 2.4 , here it is only necessary to specify the file size to be transferred.

- Erase the FLASH-SNOR, then press **Y** and hit **Enter**

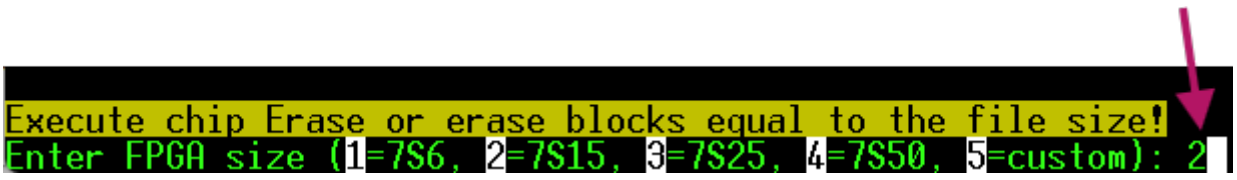


- The FLASH is now empty

- Now we can load the binary file into the FLASH-SNOR → press **10** and hit **Enter**



At this stage, a reminder will appear to erase the FLASH before writing.
Select the FPGA mounted on the board (**S715**) by typing **2** and pressing **Enter**.



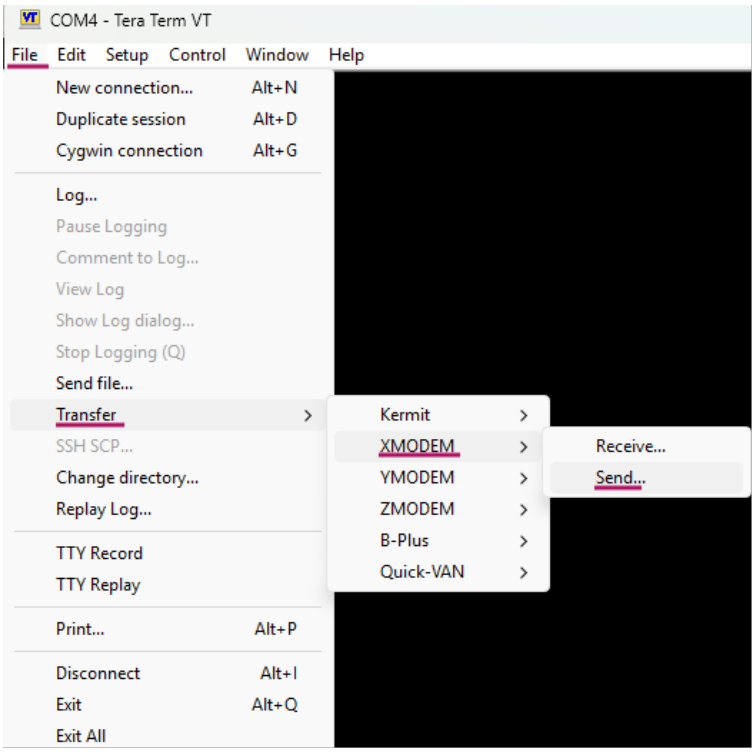
- Enter the start address for writing into FLASH-SNOR → in this case **0**

```
Enter FPGA size (1=7S6, 2=7S15, 3=7S25, 4=7S50, 5=custom): 2
Parsed file size byte: 538844
Enter start address (hex): 0
```

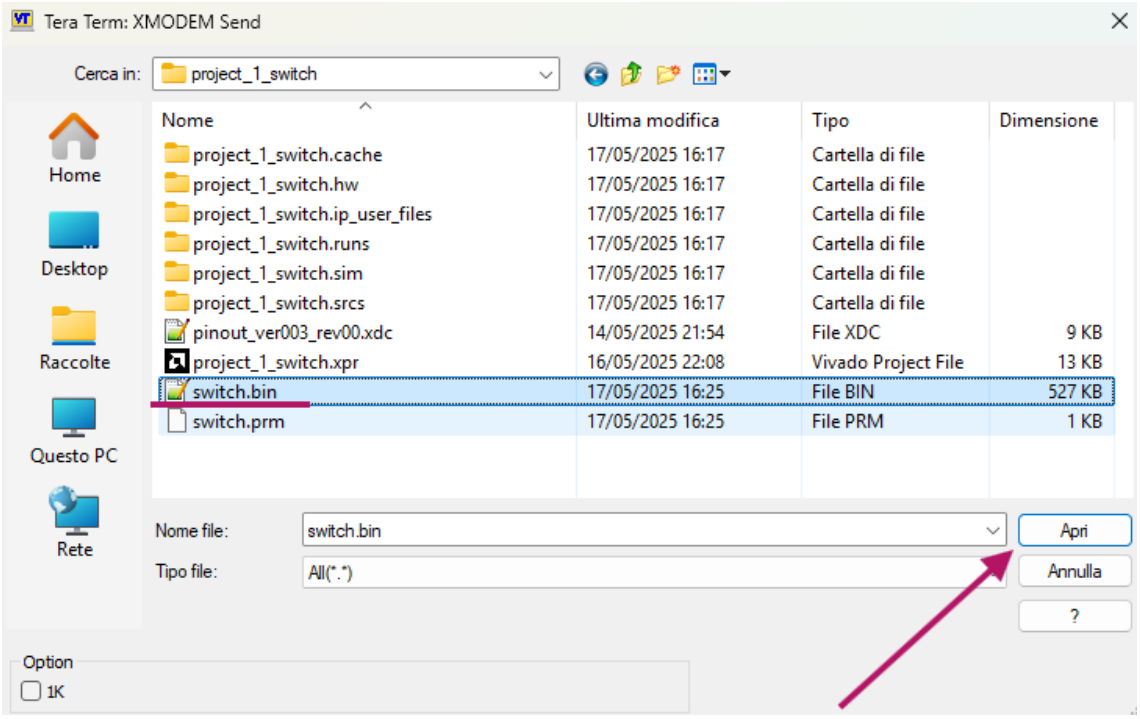
The system is now ready to receive the file

```
Enter start address (hex): 0

 Do not press any button on keyboard!
Ready to receive...
```

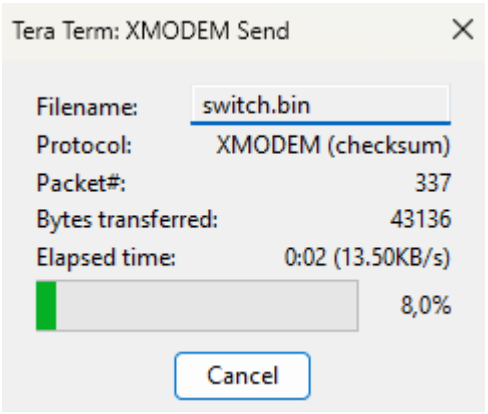
- In Tera Term go to **File → Transfer → XMODEM → Send…** and select the binary file:
  D:\Vivado_project\project_1_switch\project_1_switch.runs\impl_1\switch.bin
  or from GitHub:
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bin



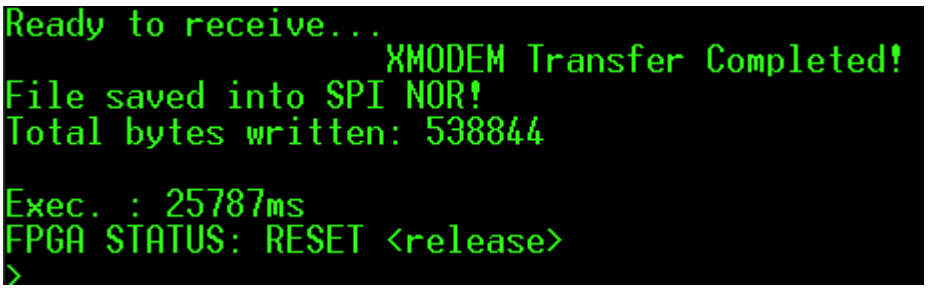
- Select the file *switch.bin*



- A window will appear showing the upload progress



- Once the binary has been written into the **FLASH-SNOR**, the FPGA will be programmed! **"ON FIRE"**



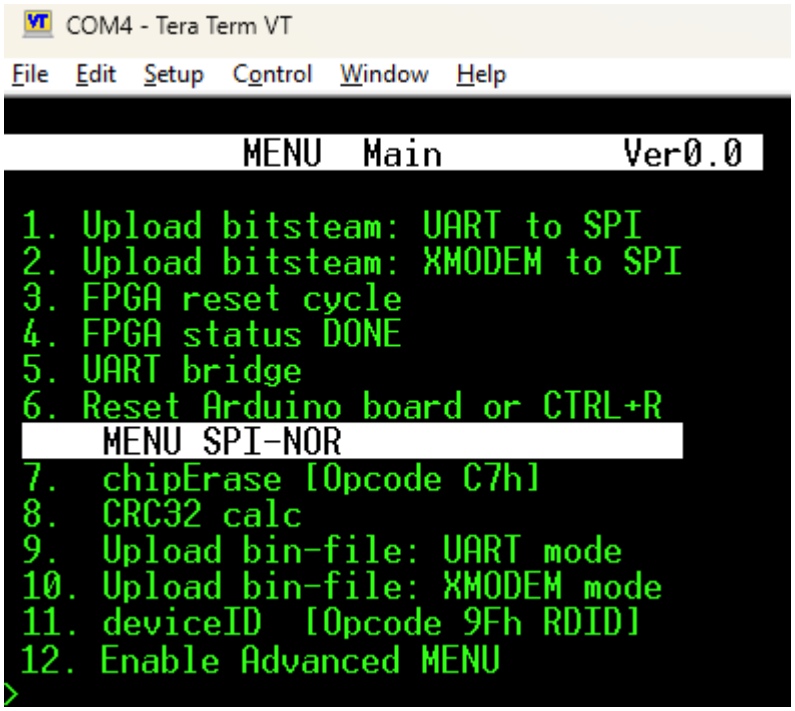
Alternatively, to check the FPGA status, select option **4** from the menu.

Chapter 3
Guide to the numeric menu options in the CLI

This chapter provides a detailed guide to the software developed on the Arduino platform.
Each available function will be explained following the order of the numeric menu implemented in the interface.

The program for Arduino R4 is available at the following repository:
https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/software/FPGA_Shield-cli.ino

After uploading the program to Arduino, connect with **Tera Term** (see 4.3 for configuration).



General operation of the CLI

The program presents itself as a numeric text menu.

- Pressing **Enter** will reprint the menu, so that the updated list of available options is always visible.
- Pressing **Ctrl+C** resets Arduino in most situations, interrupting the execution in progress.

Menu structure

The options are grouped into two main sections:

- **MENU Main** → contains the main functions for configuring and using the FPGA.
- **MENU SPI-NOR** → includes the functions dedicated to the FLASH-SNOR integrated in the **"FPGA SPARTAN7 MODULE"**, as well as some debug options.

To communication with the SNOR-FLASH memory, specific **Opcodes** are used, which are standard commands defined for this type of memory.

Communication with the SNOR-FLASH memory

- Communication between Arduino and the memory takes place via **SPI**.
- The same SPI is shared with the FPGA when it operates in **Master Mode** (FLASH jumper closed).
- To avoid access conflicts, during SNOR reading the FPGA is placed in **reset**.
- Per evitare conflitti di accesso, durante la lettura della SNOR la FPGA viene messa in **reset**.
- At the end of the operation, the reset is released.
- In the CLI, messages related to **reset enable** and **reset disable** are displayed, so that the performed operations can be tracked.

In the following subchapters, a detailed description of each numeric menu option will be provided, along with practical instructions for usage.

3.1 "1. Upload bitsteam: UART to SPI"

This option allows uploading a **bitstream file** to the FPGA.
The UART port of Arduino is used to send the bitstream file. The FPGA is programmed through the SPI serial interface, while the developed software converts the incoming UART (COMx) data into the SPI protocol.

➢ Remember: after an FPGA reset or power-off, the uploaded bitstream will be lost (it is not retained in memory).

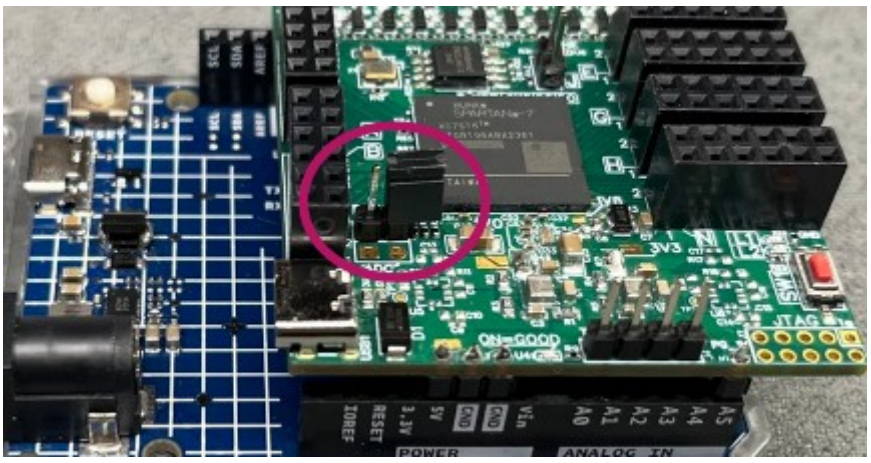
The file stored on the PC is sent to Arduino via serial, and Arduino converts the UART serial data into SPI data.
This function is supported by **Tera Term**, since it satisfies two key requirements:

1- Use of **XON/XOFF Flow Control**
2- Transmission of a maximum of **256 bytes at a time**

The **"MODULO FPGA SPARTAN7"** must be in **Slave-SPI mode**, otherwise the operation will be automatically aborted with the following message:

```
> 1
FPGA STATUS: RESET <hold>
FPGA STATUS: RESET <release>
FPGA STATUS: DONE <LOW,not programmed>
! FPGA in MASTER MODE remove jumper FLASH. Aborting...
>
```

To configure the module in **Slave-SPI**, the **jumper must be opened**.

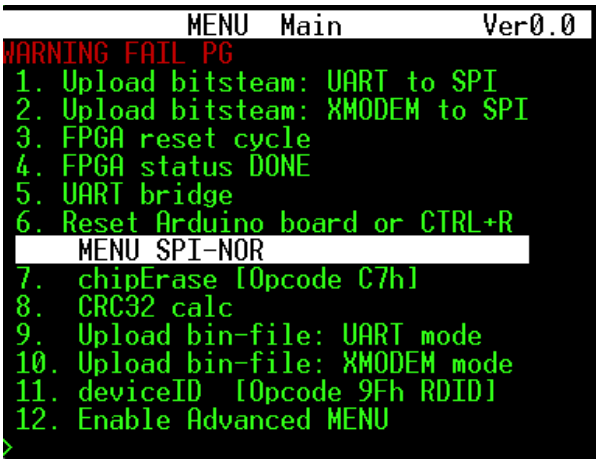


All step-by-step instructions for this procedure can be found in chapter 2.2.

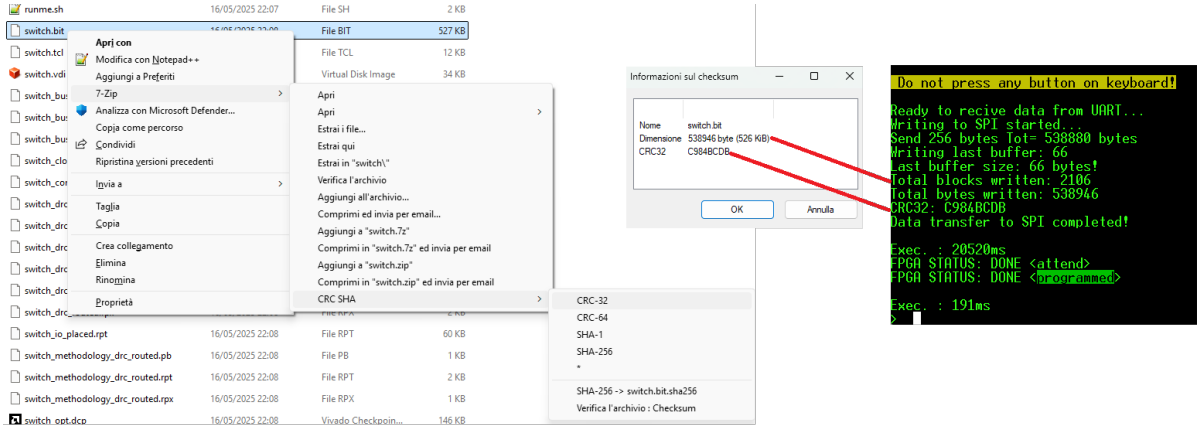
Possible issues:

- Do not use USB-C cables that are too long.
- In case of power problems, the **"FPGA SPARTAN7 MODULE"** will send an alarm signal through the **Power Good (PG)** line connected to pin A5 of Arduino.
  This can happen if the current consumption exceeds the maximum supply available from the USB port.
  Keep in mind that in more advanced projects the FPGA requires more current, and it is recommended to power the module through its dedicated USB-C port.

- If a **PG FAIL** status occurs, it will be recorded in Arduino and printed together with the MENU. The message will be cleared after an Arduino reset.



- To check if the data received by Arduino is correct: at the end of programming, the **File size** and the **CRC32 checksum** of the file will be displayed.
  Compare the CRC32 value with the one computed by **7-Zip** on your PC to verify integrity.



3.2 "2. Upload bitsteam: XMODEM to SPI"

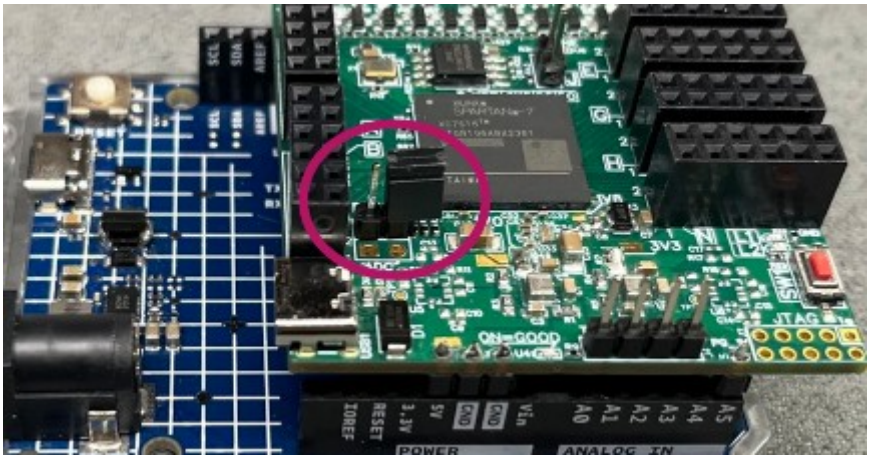
This option allows uploading a **bitstream file** to the FPGA via UART and writing it directly to the FPGA through SPI.
Remember: after an FPGA reset, the FPGA will be deprogrammed.

The file on the PC is sent to Arduino via serial, and Arduino converts the UART serial data into SPI data. We will use **Tera Term**, since it provides **XMODEM transfer**, which checks the integrity of transmitted data using a checksum (not as reliable as a CRC32).

The "MODULO FPGA SPARTAN7" must be in **Slave-SPI mode**, otherwise the operation will be utomatically aborted with the following message:

```
> 1
FPGA STATUS: RESET <hold>
FPGA STATUS: RESET <release>
FPGA STATUS: DONE <LOW,not programmed>
! FPGA in MASTER MODE remove jumper FLASH. Aborting...
```

To configure the module in **Slave-SPI**, the **jumper must be opened**.

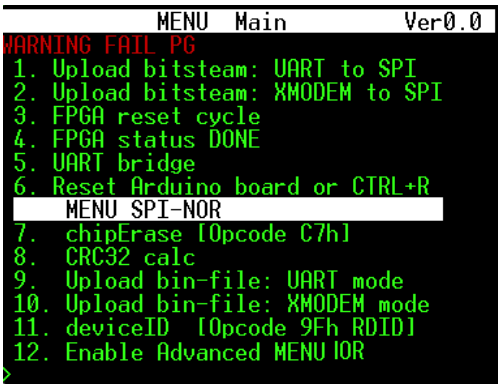


All step-by-step instructions for this procedure can be found in chapter 2.3 .

Possible issues:

- Do not use USB-C cables that are too long.
- In case of power issues, the "MODULO FPGA SPARTAN7" will send an alarm signal via the **Power Good (PG)** line connected to pin A5 of Arduino.
  This can happen if the current consumption exceeds the supply capability of the USB port.
  Remember: in more advanced projects, the FPGA may require significant current, so it is recommended to power the module through its dedicated USB-C port.

- If a **PG FAIL** status occurs, it will be recorded in Arduino and printed together with the MENU.
  The message will be cleared after resetting Arduino.



3.3 "3. FPGA reset cycle"

With command number **3**, a reset of the **"MODULO FPGA SPARTAN7"** will be performed through pin **D9 (PB)** and then released.
If the module is in **Master-SPI mode** and a binary file is present in the **FLASH-SNOR**, the FPGA will be programmed; otherwise, it will not.

3.4 "4. FPGA status DONE"

The FPGA status is reported by pin **D7 (DONE)** of the **"MODULO FPGA SPARTAN7"** .

If it is **programmed**: :

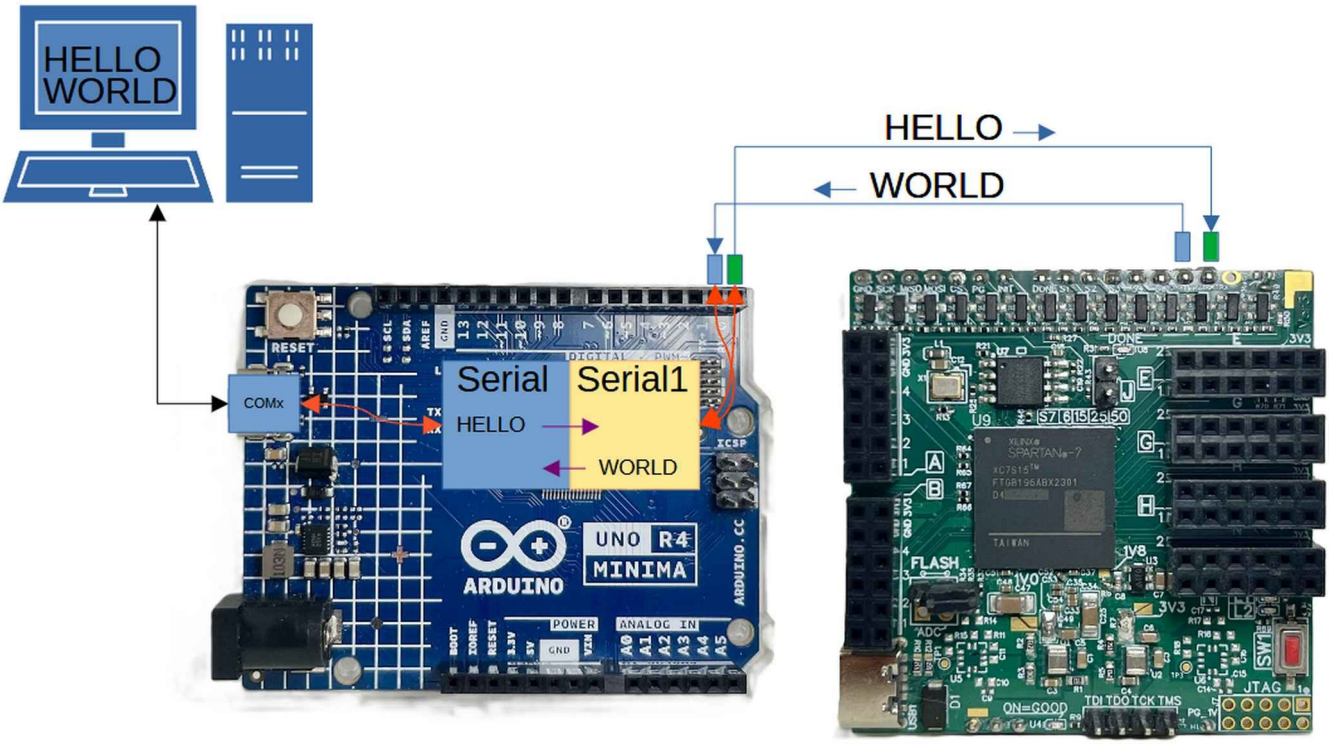
If it is **not programmed**:

3.5 "5. UART bridge"

On the "MODULO FPGA SPARTAN7"there are two pins, **UARTRX** and **UARTTX**, which are connected to Arduino pins **D0** and **D1**, natively corresponding to **SERIAL1**.

Arduino pins D0 and D1 are UART (SERIAL1) signals independent from the USB-C to UART (SERIAL) port connected to the PC.
In this mode, the **MENU/CLI** will no longer be available, and data will be forwarded between the **UARTRX / UARTTX** pins of the "MODULO FPGA SPARTAN7" and the PC's USB-C UART.
The baud rate for both ports is **115200**.

This mode is useful for communicating with or receiving UART messages generated by the FPGA (the FPGA must have a project loaded that generates such data).



To exit this mode, press the key combination **Ctrl+C**.

3.6 "6. Reset Arduino board or CTRL+R"

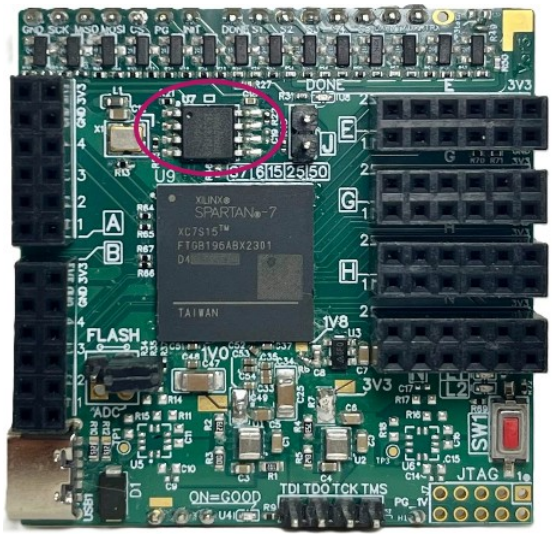
To reset Arduino R4, you can either use the **MENU option** or the **CTRL+R** key combination.
The reset command uses Arduino's `NVIC_SystemReset()` function for the Renesas core.

```
> 6

Resetting Arduino...
```

⚠ The **CTRL+R** combination is disabled during **SEND** and **RECEIVE** operations over UART.

3.7 "7. chipErase [Opcode C7h]"

This function refers to the SNOR-FLASH present on the "**MODULO FPGA SPARTAN7**".
It allows you to erase the entire memory and must be executed **before** using the write functions described in Chapters 2.4 e 2.5 .



- Once executed, the FLASH is now empty.

```
FPGA STATUS: RESET <hold>
 You want erase whole chip [Y/N]: y
Chip Erase in progress...
.................
Chip Erase completed!

Exec. : 8000 ms
FPGA STATUS: RESET <release>
>
```

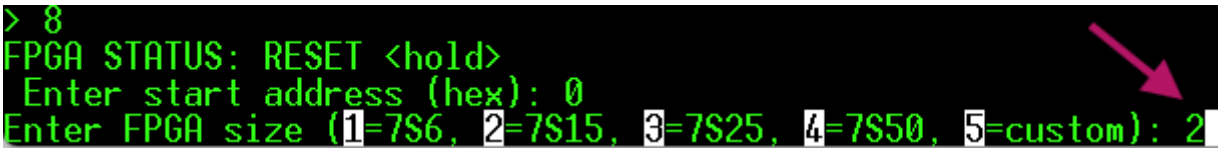
3.8 "8. CRC32 calc"

This function calculates the **CRC32** of a selected section of SNOR-FLASH memory, where the FPGA binary is typically stored (see Chapters 2.4 e 2.5).

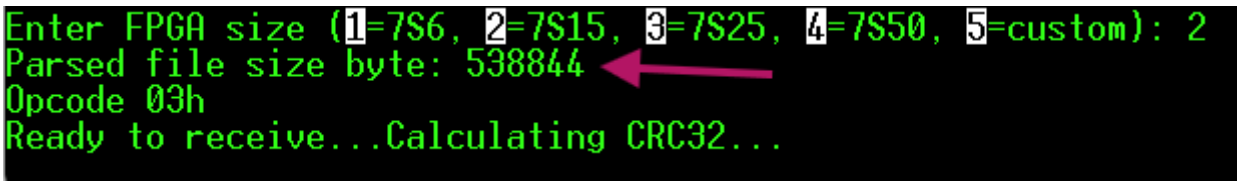
It is useful for debugging to verify the integrity of the binary file written to the FLASH-SNOR. The calculated CRC32 can then be compared with the CRC32 of the file on the PC using tools like **7-Zip**.

- Select the start address (default = 0).

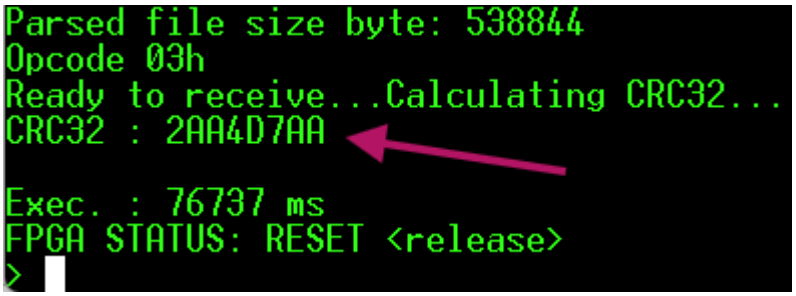
- Define the file size to be read. If the size is unknown, you can use the suggested defaults.

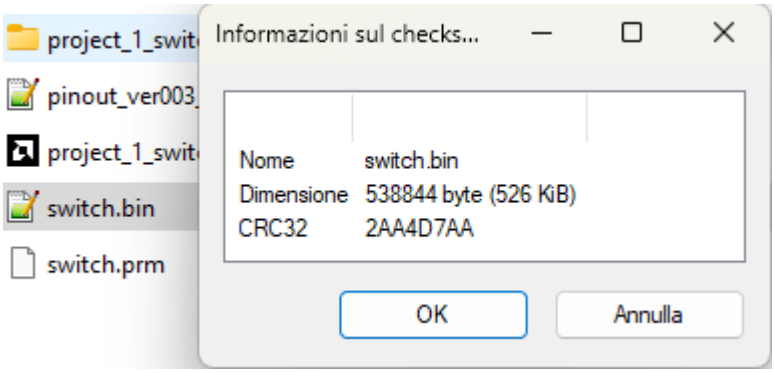


- The function will indicate the file size being compared. Execution time is about **76 seconds for 500 KB**.



- Finally, compare the calculated CRC32 with the CRC32 of the file stored on your PC using **7-Zip**.





Communication details:

Communication between Arduino and the FLASH-SNOR occurs via SPI.
This is the same SPI used by the FPGA when operating in Master Mode (jumper “FLASH” closed). To avoid access conflicts, the FPGA is kept under reset during FLASH read operations, and reset is released only after the operation is completed.

3.9 "9. Upload bin-file: UART mode"

As mentioned earlier, the **"MODULO FPGA SPARTAN7"** includes a non-volatile **FLASH-SNOR memory**, where you can write your custom binary file.

The binary file stored on the PC will be sent to Arduino via UART. Arduino then converts the UART serial data into SPI commands for writing to the FLASH-SNOR. In this example, we will use **Tera Term**, since it satisfies two essential requirements:

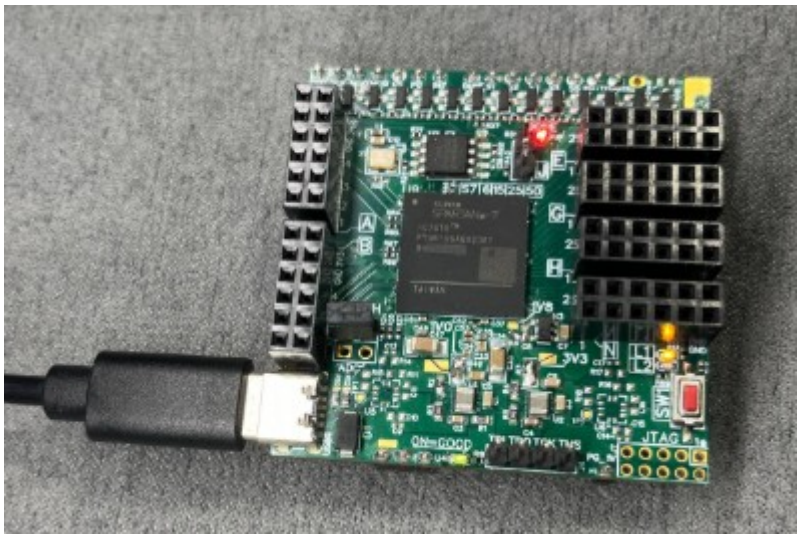
1. Support for **XON/XOFF flow control**.
2. Transmission limited to **256 bytes per block**.

Once the binary file has been successfully written to FLASH-SNOR, you must configure the module in **Master SPI Mode**. In this mode, the FPGA automatically loads the binary at power-up or reset, making the "MODULO FPGA SPARTAN7" fully **stand-alone**.

⚠ It is important to erase the FLASH-SNOR before every new binary upload.

The full step-by-step procedure is described in **Chapter 2.4**.

After programming, you can use the board without Arduino by simply powering it through the **USB-C port**.



Alternatively, if you continue to use the "MODULO FPGA SPARTAN7" with Arduino, you can force the FPGA to load from FLASH-SNOR by closing the **“FLASH” jumper** and performing a reset cycle (see 3.3).

3.10 "10. Upload bin-file: XMODEM mode"

The **"MODULO FPGA SPARTAN7"**, as previously mentioned, includes a non-volatile **FLASH-SNOR memory** where you can write your binary file.

The file stored on the PC will be sent to Arduino via UART, and Arduino will convert the UART serial data into SPI commands. In this example, we will use **Tera Term**, which provides **XMODEM transfer** support. This is an alternative method, as described in **Chapter 3.9.**.

⚠ It is important to erase the FLASH-SNOR before every new binary upload.

The complete step-by-step procedure is described in **Chapter** 2.5.

After programming, you can use the board without Arduino by simply powering it through the **USB-C port**.

Alternatively, if the **"MODULO FPGA SPARTAN7"** is used together with Arduino, you can force the loading from FLASH-SNOR by closing the **“FLASH” jumper** and performing a reset cycle (see 3.3).

3.11 "11. deviceID [Opcode 9Fh RDID]"

To verify proper communication between Arduino and the **"MODULO FPGA SPARTAN7"** , you can check the **device ID** of the FLASH-SNOR present on the module.
This function is intended only for **debug purposes** and will not be used in other procedures.

As you may have noticed, the **Opcodes** used for communication are also shown — these are simply standard commands for FLASH-SNOR devices.

- Command **11** → Read SNOR device ID

```
> 11
FPGA STATUS: RESET <hold>
Manufacturer ID: 0xEF
Memory Type: 0x40
Memory Capacity: 0x16
FPGA STATUS: RESET <release>
>
```

3.12 "12. Enable Advanced SPI-NOR menu"

There are additional hidden debug functions that can be enabled using CLI option **"12"**. These functions are intended for **debugging or advanced use** and are detailed in the following chapters.

- Once the debug functions are enabled, the **MENU** will display new options:

```
> 12
 Advanced MENU: Enabled
>
              MENU  Main            Ver0.0

 1. Upload bitsteam: UART to SPI
 2. Upload bitsteam: XMODEM to SPI
 3. FPGA reset cycle
 4. FPGA status DONE
 5. UART bridge
 6. Reset Arduino board or CTRL+R
     MENU SPI-NOR
 7.  chipErase [Opcode C7h]
 8.  CRC32 calc
 9.  Upload bin-file: UART mode
 10. Upload bin-file: XMODEM mode
 11. deviceID  [Opcode 9Fh RDID]
 12. Enable Advanced MENU
       MENU Advanced
 13. Set SPI
 14. blockErase 4096/32768/65536 [Opcode 20h/52h/D8h]
 15. writeEnable [Opcode 06h WREN]
 16. readFlash [Opcode 03h RDSR]
 17. fast-read Flash [Opcode 0Bh FRD]
 18. statusRegister [Opcode=0x05 RDSR]
 19. writeData [Opcode 02h PP]
 20. deviceID  [Opcode 90h REMS]
 21. Download SNOR-Flash: UART mode
 22. Download SNOR-Flash: XMODEM mode
 23. Custom SPI Transaction
 24. FPGA Reset: Hold
 25. FPGA Reset: Realese
>
```

- To disable the **"Advanced MENU"**, send **12** in the CLI.

```
> 12
 Advanced MENU: Disabled
>
```

3.13 "13. Set SPI"

Two types of SPI are used:

- **Hardware SPI** during bitstream upload (options 3.1 and 3.2).
- **Software SPI** during bin-file upload to FLASH-SNOR.

The only configurable SPI is the **Hardware SPI**.

The default Hardware SPI configuration is as follows:

- **Speed**: 2,000,000 Hz
- **Bit order**: MSBFIRST
- **Mode**: SPI_MODE0 [CPOL=0, CPHA=0]

Setting:

- Let's try changing the SPI configuration. After an Arduino reset, the configuration returns to its default. In this example, we will only change the clock frequency to the maximum measured on Arduino R4 Minima (12 MHz).

- Set the bit order to **MSBFIRST**.

- Finally, set the sampling mode to **SPI_MODE0**.

Increasing the SPI speed to **12 MHz** reduces bitstream upload time to **12.5 seconds**, which is not a major advantage compared to **20 seconds**. For reliability, it is recommended to keep the default setting of **2 MHz**.

3.14 14. blockErase 4096/32768/65536 [Opcode 20h/52h/D8h]"

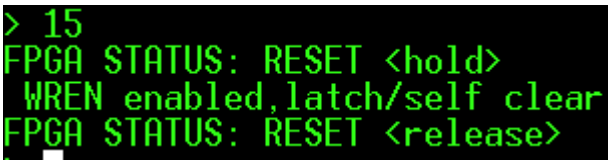
Partial erase of the FLASH-SNOR has also been implemented, allowing you to select:

- the **start address**,
- the **erase size type**,
- and the **number of blocks** to erase.

To verify the memory contents before and after the block erase, use the option described in **Chapter 3.16**.

3.15 "15. writeEnable [Opcode 06h WREN]"

The FLASH-SNOR devices used include a **write-enable register** that prevents unintentional writes. In standard procedures (outside of debug operations), this register is managed **automatically**.

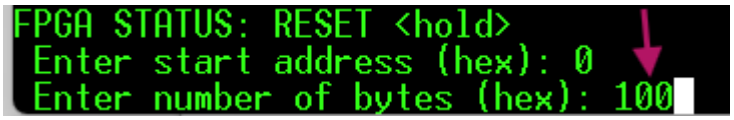


3.16 "16. readFlash [Opcode 03h RDSR]"

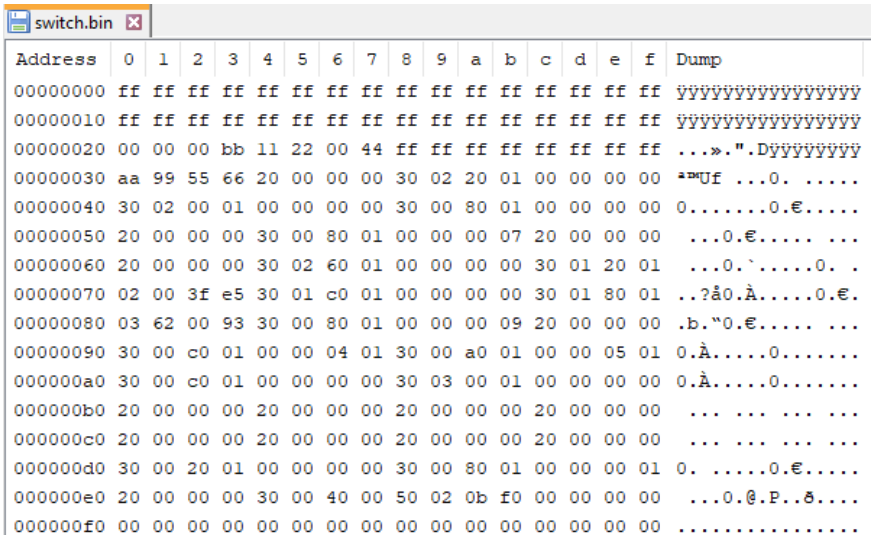
The debug option for reading the contents of the FLASH-SNOR can be useful to compare the file stored on your PC with the file written in FLASH-SNOR.

- Select the **start address** of the SNOR (⚠ values are interpreted as hexadecimal).

- Define the **number of bytes** to read (also in hexadecimal).



- The output can then be compared with the file stored on the PC (for example, using **Notepad++** with the **HEX-Viewer** plugin).



3.17 "17. fast-read Flash [Opcode 0Bh FRD"

The debug option for reading the contents of the FLASH-SNOR can be useful to compare the file stored on your PC with the file written in FLASH-SNOR.

This function is similar to **Chapter** 3.16, but it uses a **different Opcode** — the same one used by the FPGA when operating in **Master SPI** mode.

3.18 "18. statusRegister [Opcode=0x05 RDSR]"

The **status register** is an internal register of the FLASH-SNOR that describes the various states of the memory.
For example, during a write operation, the status register can be read to check whether the writing process has completed.

```
> 18
FPGA STATUS: RESET <hold>
Status Register: 0x2
Write Enabled
FPGA STATUS: RESET <release>
>
```

Below is an excerpt from the datasheet:

7.1 Status Registers

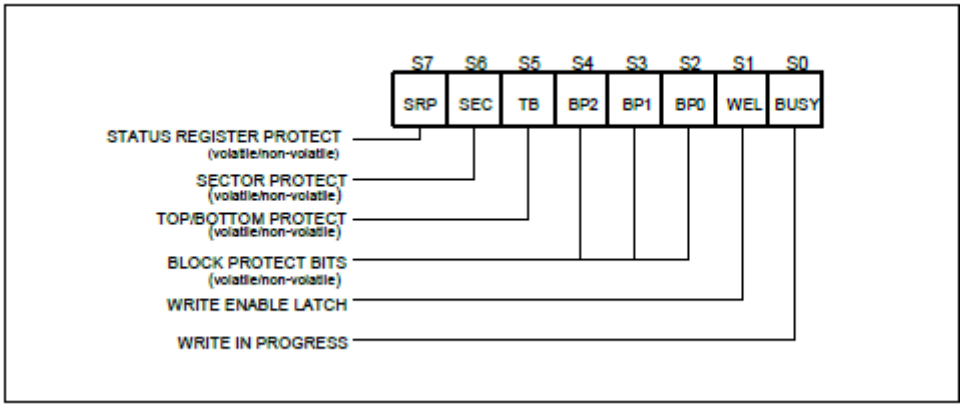


Figure 4a. Status Register-1

3.19 "19. writeData [Opcode 02h PP]"

To write a single byte, it is first necessary to erase the memory.
The **minimum erasable size** is one block (see 3.13), otherwise the entire SNOR FLASH chip can be erased.

To verify whether a sector has been erased, the byte must first be read:

- if the value is **FF**, it means the location is ready for writing.

In this chapter, we will use the functions explained in the previous chapters.
The goal of this example is to write a few bytes into the FLASH-SNOR.

- Read the first **48 bytes** of the FLASH.

```
> 16
FPGA STATUS: RESET <hold>
 Enter start address (hex): 0
 Enter number of bytes (hex): 30
 Reading SNOR:
0x00000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF   |................|
0x00010: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF   |................|
0x00020: 00 00 00 BB 11 22 00 44 FF FF FF FF FF FF FF FF   |.....".D........|
 Read complete!

Exec. : 14 ms
FPGA STATUS: RESET <release>
>
```

- **Erase** the first sector of the FLASH-SNOR.

```
> 14
FPGA STATUS: RESET <hold>
 Enter start address (hex): 0
 Enter block size (4096, 32768, 65536): 4096
 Enter number of blocks: 1
Block erase 4KB from address 0x0
All blocks are erased!

Exec. : 101 ms
FPGA STATUS: RESET <release>
>
```

- Read again the first **48 bytes**: you will see they are all **FF** up to byte **4095**.

```
> 16
FPGA STATUS: RESET <hold>
 Enter start address (hex): 0
 Enter number of bytes (hex): 30
 Reading SNOR:
0x00000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF   |................|
0x00010: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF   |................|
0x00020: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF   |................|
 Read complete!

Exec. : 14 ms
FPGA STATUS: RESET <release>
>
```

- Now it is possible to write one byte at a time.

```
> 19
FPGA STATUS: RESET <hold>
Erase block before write!
 Enter start address (hex): 0
 Enter data byte (hex): 43
 Byte written to address 0x0: 0x43
```

```
> 19
FPGA STATUS: RESET <hold>
Erase block before write!
 Enter start address (hex): 1
 Enter data byte (hex): 49
 Byte written to address 0x1: 0x49
```

```
> 19
FPGA STATUS: RESET <hold>
Erase block before write!
 Enter start address (hex): 2
 Enter data byte (hex): 41
 Byte written to address 0x2: 0x41
```

```
> 19
FPGA STATUS: RESET <hold>
Erase block before write!
 Enter start address (hex): 3
 Enter data byte (hex): 4f
 Byte written to address 0x3: 0x4F
```

⚠ **Warning**: if you make a mistake while writing, you must erase the entire sector.

- Read again the first **16 bytes**, you have just written your first word.

```
> 16
FPGA STATUS: RESET <hold>
 Enter start address (hex): 0
 Enter number of bytes (hex): 10
 Reading SNOR:
0x00000: 43 49 41 4F FF FF FF FF FF FF FF FF FF FF FF FF   |CIAO............|
 Read complete!
```

- Power cycle the board and read again the first **16 bytes**!

3.20 "20. deviceID [Opcode 90h REMS"

This option is similar to **chapter** 3.11, but differs in the SPI command sent to the **FLASH-SNOR**. This function is intended for **debugging only** and will not be used in other procedures.

As you have seen, the **Opcodes** used for communication are also indicated (see the **FLASH-SNOR datasheet**), which are simply standard commands for **FLASH-SNOR**.

- Read device ID command fot SNOR

```
> 20
FPGA STATUS: RESET <hold>
Manufacturer ID: 0xEF
Device ID: 0x15
FPGA STATUS: RESET <release>
>
```

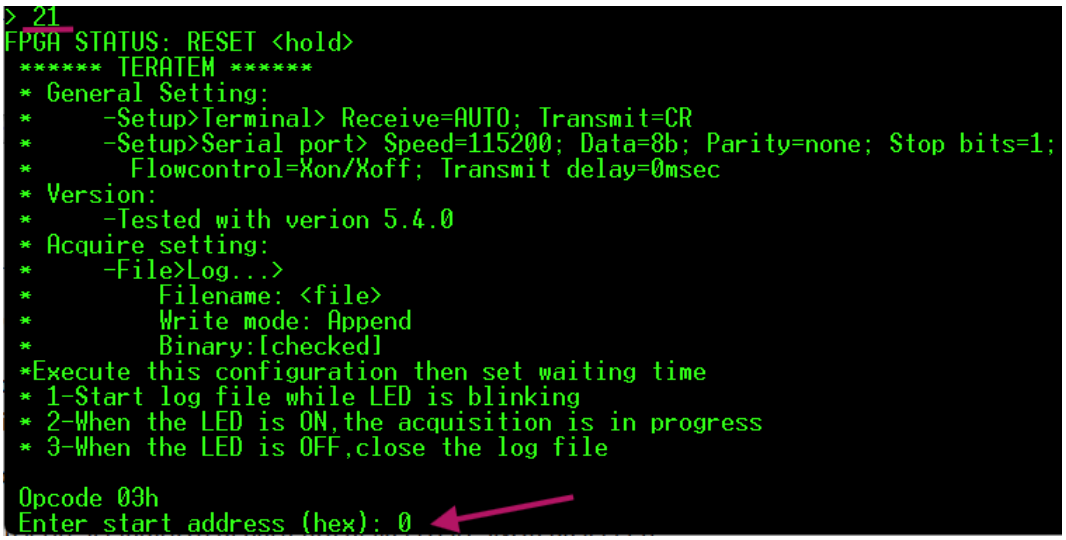
3.21 "21. Download SNOR-Flash: UART mode"

We have already seen how to write the binary to the **FLASH-SNOR**. Now, let's see how it is also possible to **read the memory content** and copy it into a file on the PC.

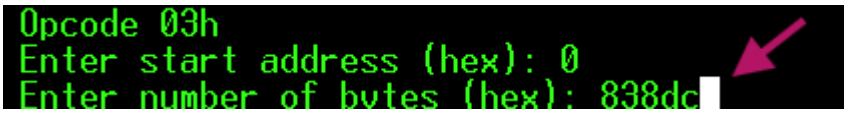
In this example, the content of the **FLASH-SNOR** is the binary file available in the repository:
https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bin

To copy the memory content, you need to use **Tera Term**.

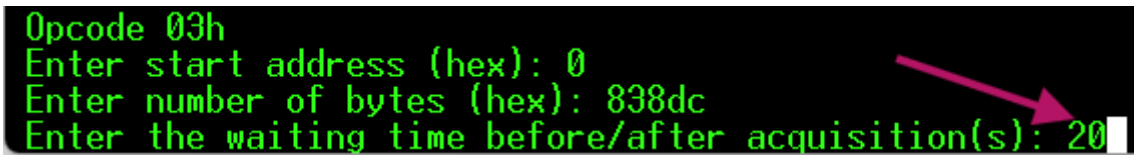
- Enter the starting address of the **FLASH-SNOR** to read.



- Enter the number of bytes to read in hexadecimal. For example, the file :
  https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bin
  has a size of **538,844 bytes**, which is **838DC** in hexadecimal.

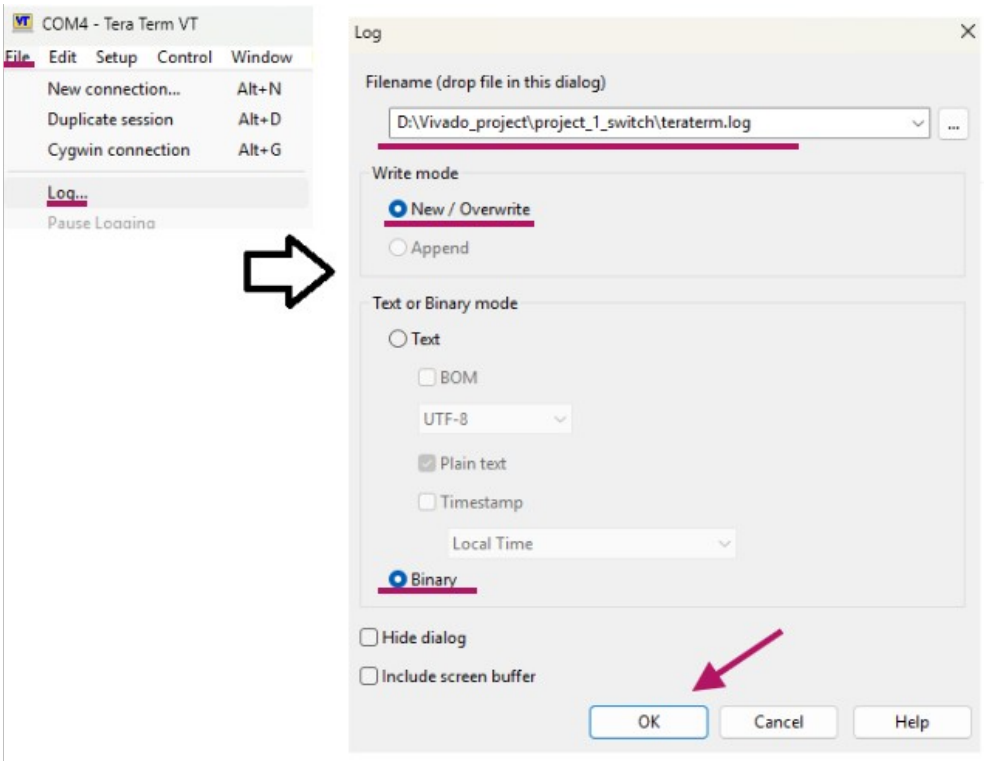


- Now enter the **waiting time** before the **FLASH-SNOR** content is transmitted over the serial port. This time is required to configure **Tera Term** for capturing the file from the console.

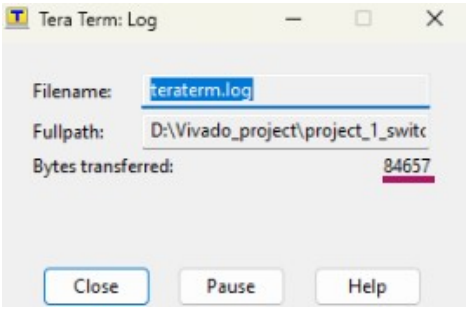


- The Arduino **D13 LED** will blink during this time. Be quick!

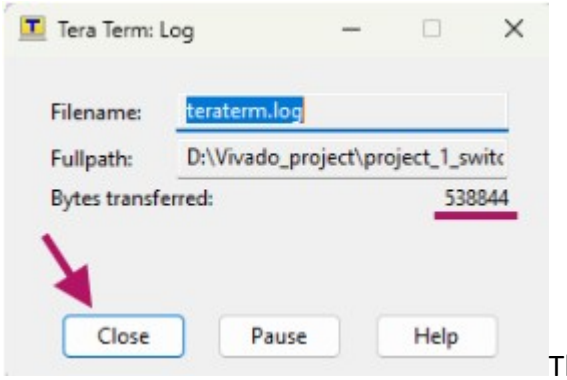
- You need to be fast in configuring **Tera Term**, the **D13 LED** is blinking in the meantime.



- At the end of the **D13 LED blinking**, the LED will turn **ON steadily**, and you will see the transfer window incrementing.



When the **D13 LED** turns off, you will have the same amount of time previously set to close the **.log file**.



The transferred bytes will match the bytes you entered earlier.

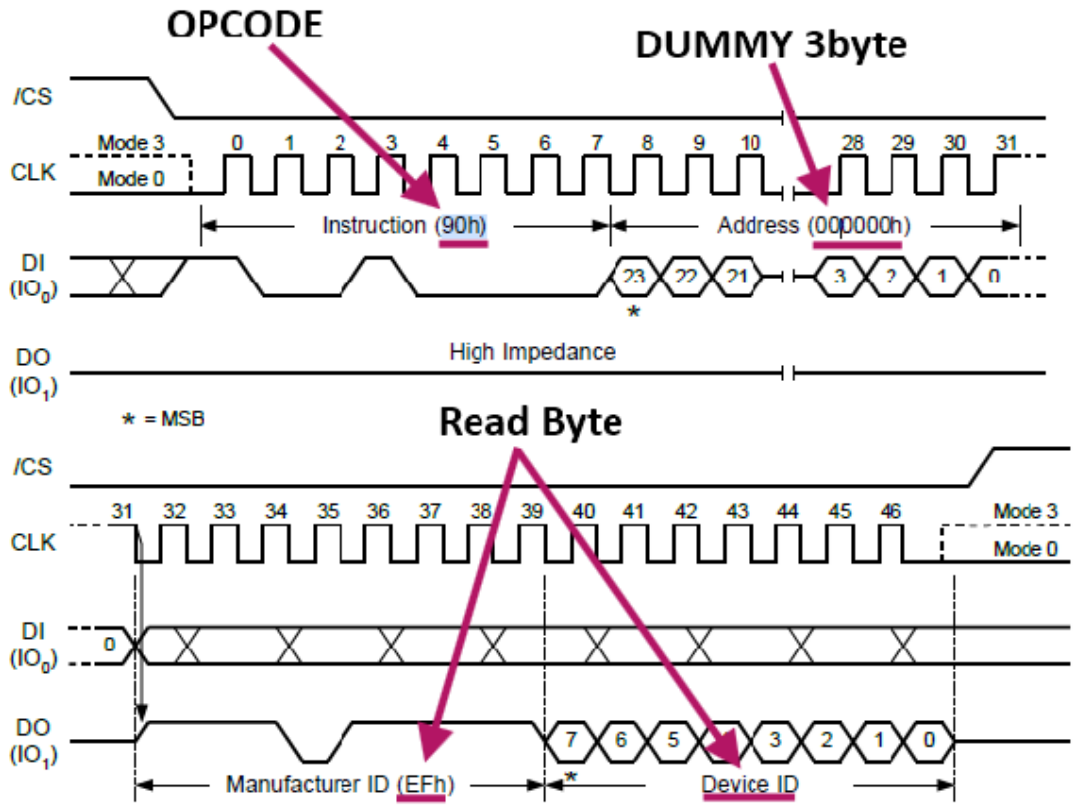
- At this point, you can compare/check the downloaded file to verify it matches the repository file using **7zip** or **Notepad++**.

3.22 "23. Custom SPI Transaction"

The **"Custom SPI"** option allows sending custom commands to the **FLASH-SNOR**, for example, to read an OPCODE not integrated in this software.

Arduino's SPI is configured as follows: **SCK = D13**, **MISO = D11**, **MOSI = D12**, and **CS = D10**.

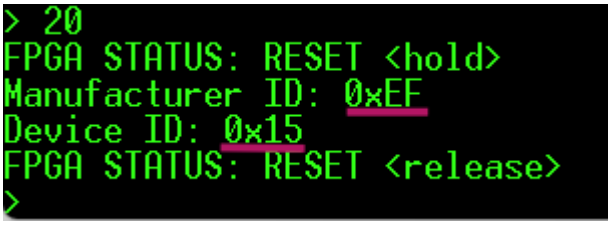
In this example, we will read the **device ID** of the **FLASH-SNOR**, which has **OPCODE 90h**. Below is the relevant excerpt from the datasheet for **OPCODE 90h**.



- Configure the CLI as described in the datasheet... it returns **0xEF** and **0x15**.

```
> 23
FPGA STATUS: RESET <hold>
Enter Write Byte (hex, e.g. opcode): 90
How many dummy bytes? 3
Dummy byte value (0=0x00, 1=0xFF): 00
How many bytes do you want to read? 2
Response: EF 15
Exec. :
1ms
FPGA STATUS: RESET <release>
>
```

- Now we can compare the value read with the CLI function.



3.23 "22. Download SNOR-Flash: XMODEM mode"

We have already seen how to write the binary to the **FLASH-SNOR**. Now, let's see how it is also possible to **read the memory content** and copy it into a file on the PC.

This procedure is similar to the one described in **chapter** 3.21 , but simplified by using the **XMODEM protocol** for file reception, so there is no need to be particularly fast in configuring **Tera Term**.

The only limitation is that it handles multiple files of at least **128 bytes** and in multiples of **128 bytes**. If the file is not a multiple of 128, additional bytes called **PADDING** are added.

In this example, the content of the **FLASH-SNOR** is the binary file available in the repository:

https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/example/switch/switch.bin
The file size is **538,844 bytes**, which is not a multiple of 128 bytes. In this case, the program adds bytes to make it a multiple of 128. On the PC, you may need to account for these bytes or remove them manually using a text editor such as **Notepad++**.

- Enter the **starting address** of the FLASH-SNOR to read.

```
> 22
FPGA STATUS: RESET <hold>
 Opcode 03h
Enter start address (hex): 0
```

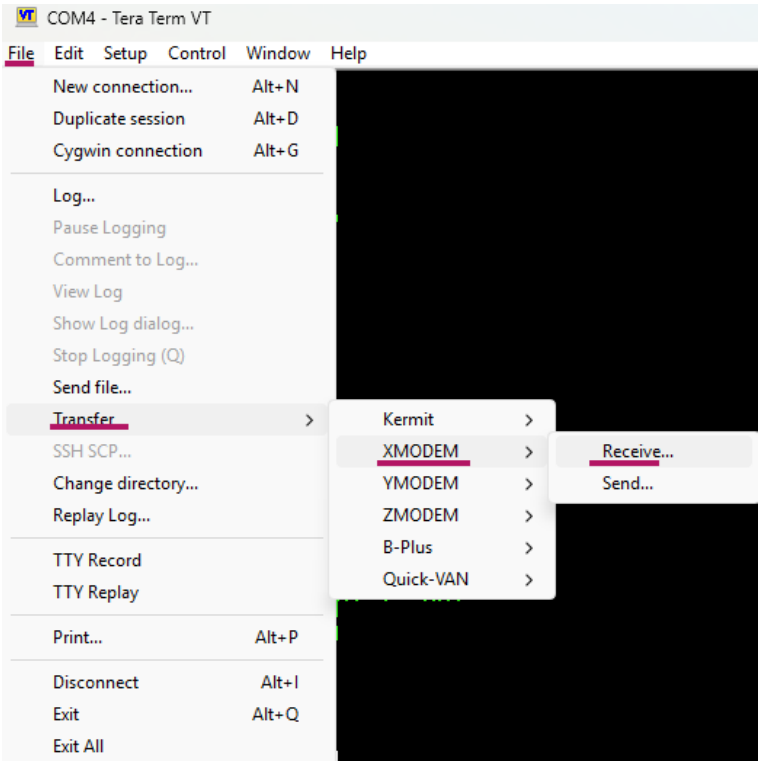
- Enter the number of bytes to read in hexadecimal, for example the file.

```
Enter start address (hex): 0
Enter FPGA size (1=7S6, 2=7S15, 3=7S25, 4=7S50, 5=custom): 2
Parsed file size byte: 538844
```

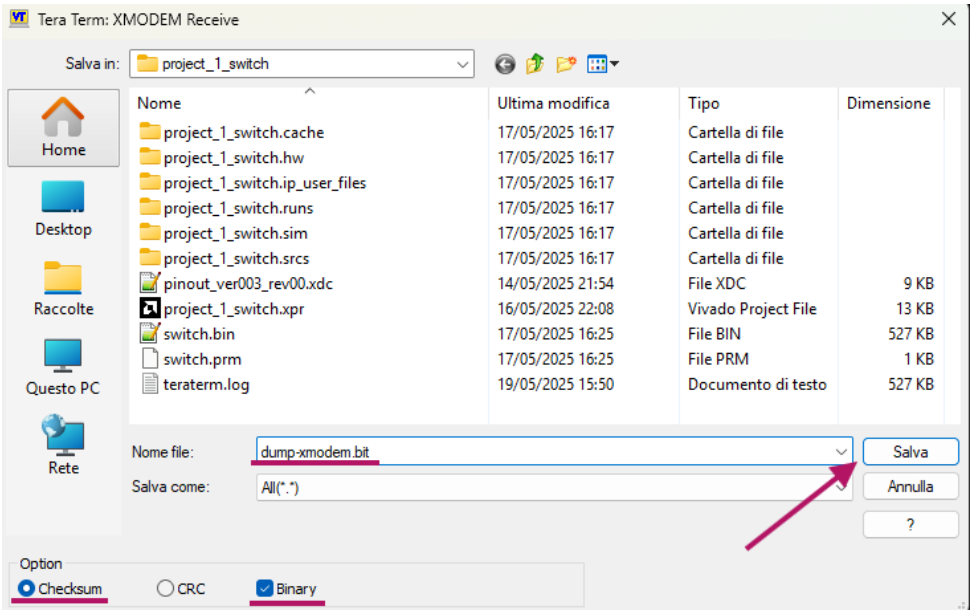
- Define the value of the **padding byte**.

```
Enter FPGA size (1=7S6, 2=7S15, 3=7S25, 4=7S50, 5=custom): 2
Parsed file size byte: 538844

Warning: File size is not a multiple of 128 bytes.
Enter padding byte (hex, default 1A): 1a
```

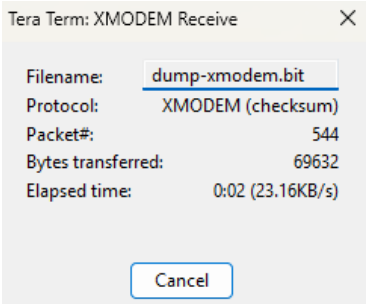
- Enable reception via the **XMODEM protocol**.



- Select where to save the file; reception will start automatically.



The acquisition will then begin.



- **END**

```
Enter padding byte (hex, default 1A): 1a
Waiting for receiver...
XMODEM Transfer Completed!
Total bytes sent: 538844

Exec. time: 22191ms
FPGA STATUS: RESET <release>
>
```

- If we compare the file **switch.bin** with the dumped file **dump-xmodem.bin**, we will notice a difference at the end of the file.

dump-xmodem.bit

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Dump
00083870	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083880	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083890	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838a0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838b0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838c0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838d0	20	00	00	00	20	00	00	00	20	00	00	00	1a	1a	1a	1a	... ...
000838e0	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	....
000838f0	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	1a	....

Normal text file | length : 538.880 lines : 3 | Ln : 1 Col : 2 Sel : 1 | 1

dump-xmodem.bit | switch.bin

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	Dump
00083850	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083860	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083870	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083880	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
00083890	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838a0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838b0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838c0	20	00	00	00	20	00	00	00	20	00	00	00	20	00	00	00	... ...
000838d0	20	00	00	00	20	00	00	00	20	00	00	00					... ...

Normal text file | length : 538.844 lines : 3 | Ln : 1 Col : 1

3.24 "24. FPGA Reset: Hold"

Set the **PG_B "PROGRAM"** pin, corresponding to **D9 on Arduino**, to **0V**. This puts the "MODULO FPGA SPARTAN7" into reset. To release the reset, see **chapter** 3.25.

- FPGA Reset **ON**

3.25 "25. FPGA Reset: Realese"

Set the **PG_B "PROGRAM"** pin, corresponding to **D9 on Arduino**, to **5V**. This releases the reset of the "**MODULO FPGA SPARTAN7**" .

- FPGA Reset **OFF**

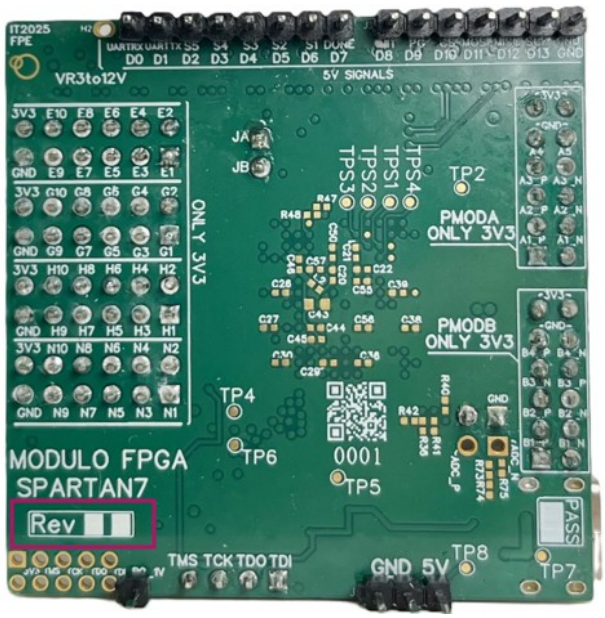
Chapter 4
Appendix

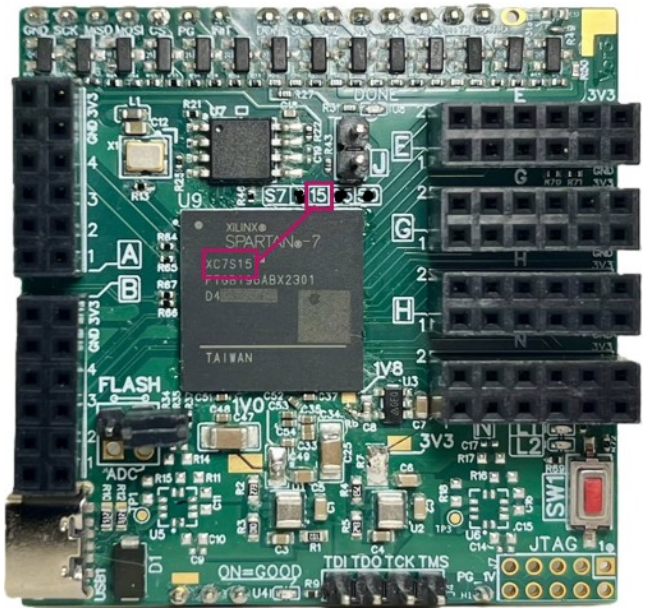
4.1 Credits

- **Tera Term**
  Copyright (C) 1994-1998 T. Teranishi
  (C) 2004-2025 TeraTerm Project
  All rights reserved.
- **Arduino ®**
  https://www.arduino.cc/en/trademark
  This project was developed using the **Arduino** platform, an open-source environment for electronic prototyping. We thank the **Arduino community** and project contributors for the documentation, software tools (**Arduino IDE**), and available hardware.
  Official website www.arduino.cc
- **Notepad++**
  https://notepad-plus-plus.org/downloads/

4.2 Module Hardware Revision

To check the board revision, it is sufficient to inspect the back of the **"MODULO FPGA SPARTAN7"** (see left image). Additionally, the **"MODULO FPGA SPARTAN7"** boards support **4 FPGA versions**, which can be seen on the front of the board (right image).





Here is a summary of the different versions:

- **Rev00**:
  

  CLK=100MHz, SNOR=W25Q32JV, FPGA=**xc7s15**ftgb196-2

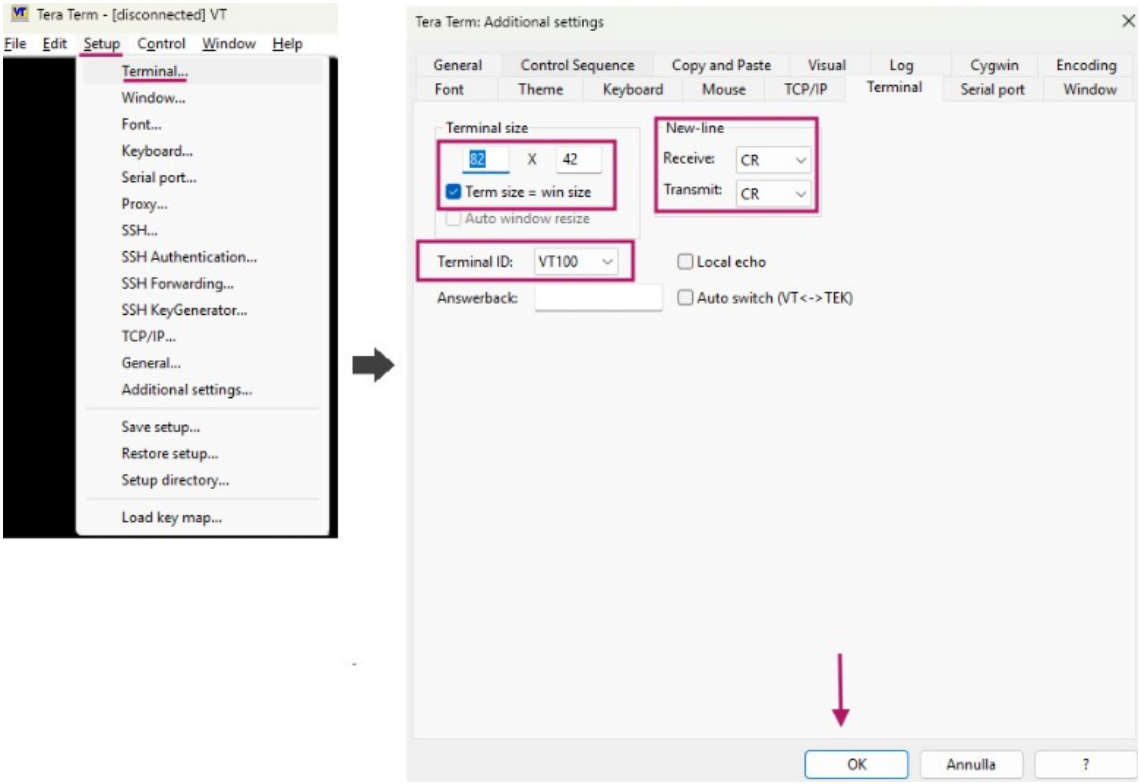
- **RevXX**:
  TBD

4.3 Configurazione Tera Term

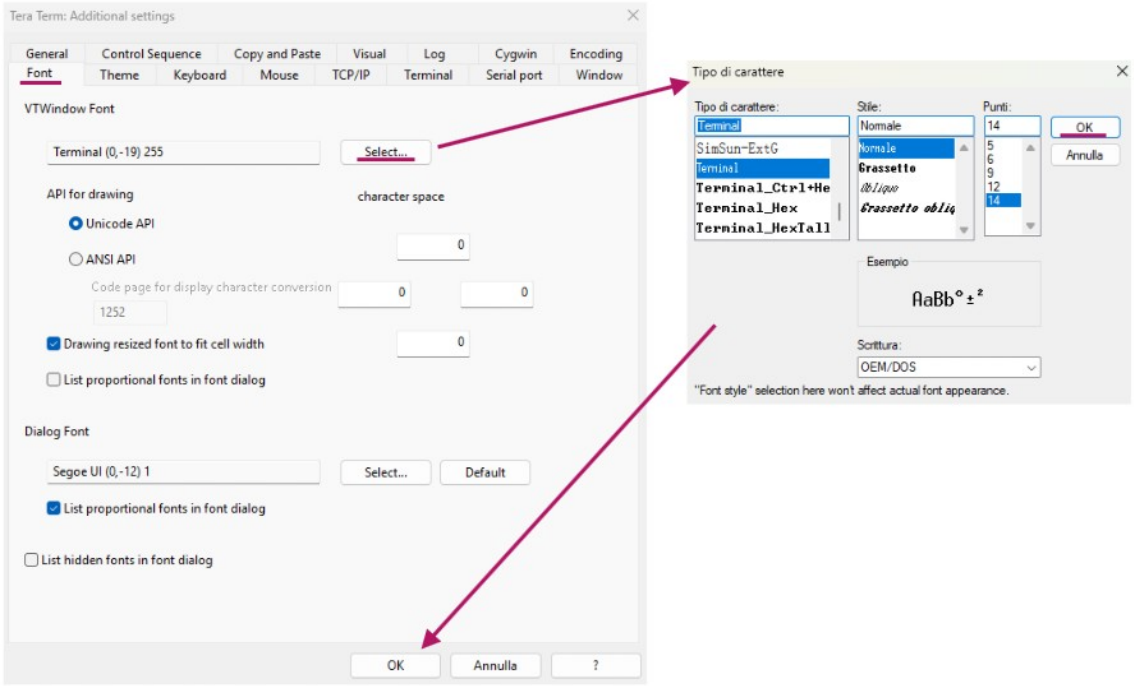
This configuration refers to **Tera Term [Version 5.4.0]**, downloadable from *https://teratermproject.github.io/*

Configure Tera Term following the steps below

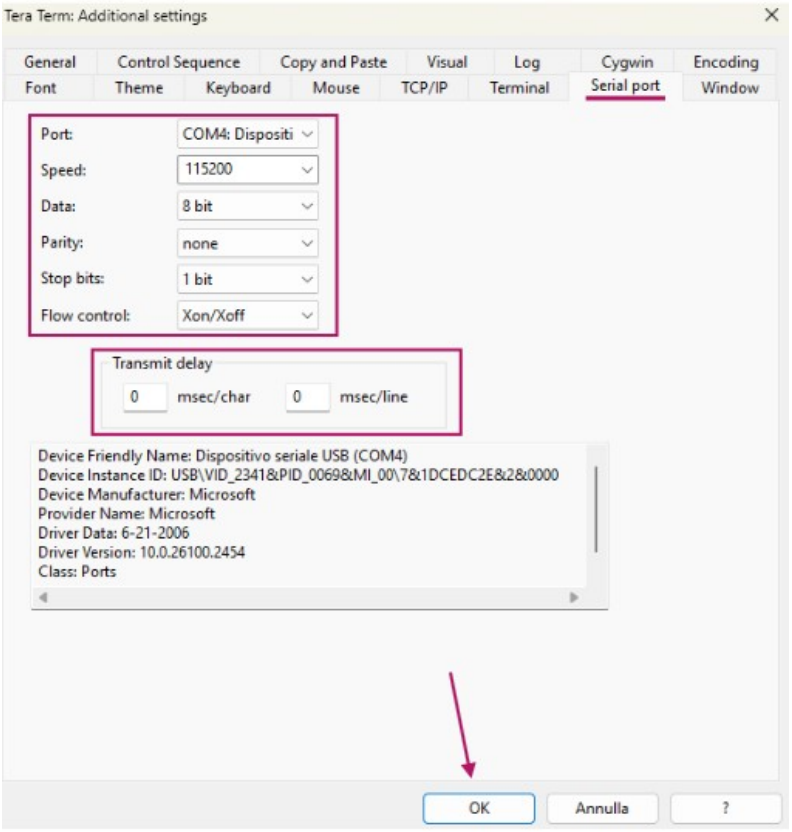
- Configure the **Terminal**.



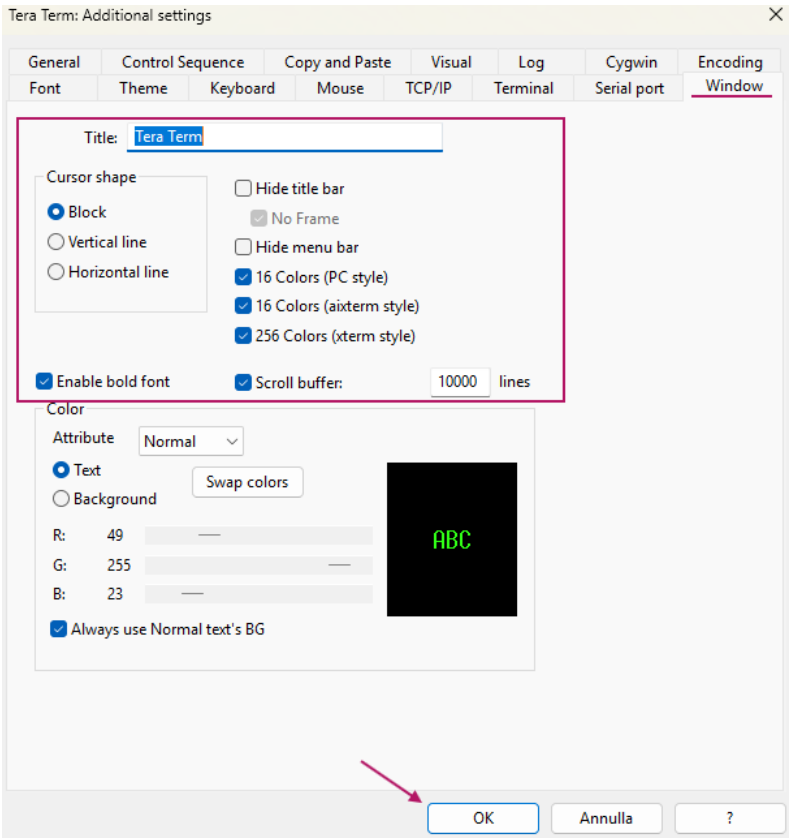
- Configure the **Font** according to your preference.



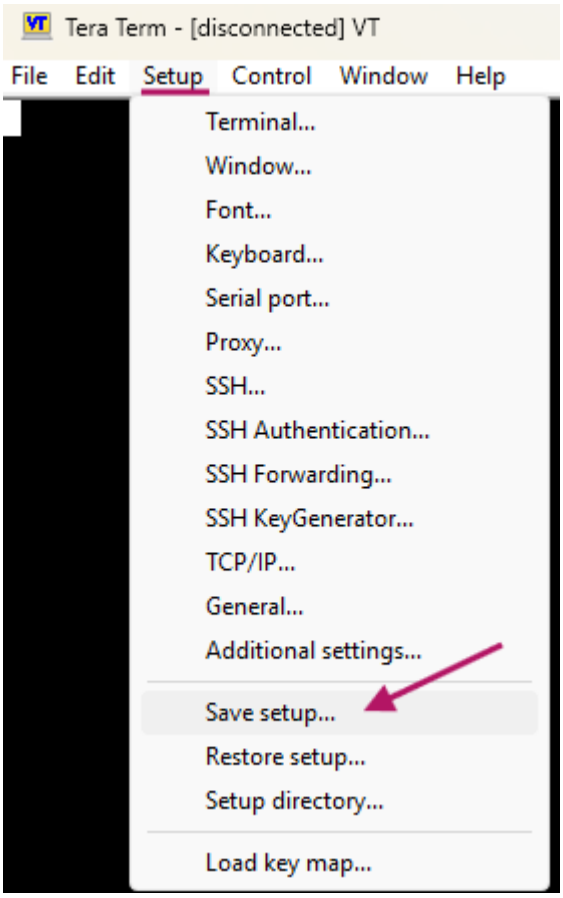
- Configure the **Serial Port**. ⚠ Make sure the COM port matches the one assigned by Windows for your PC; in my case, it is **COM4**. .



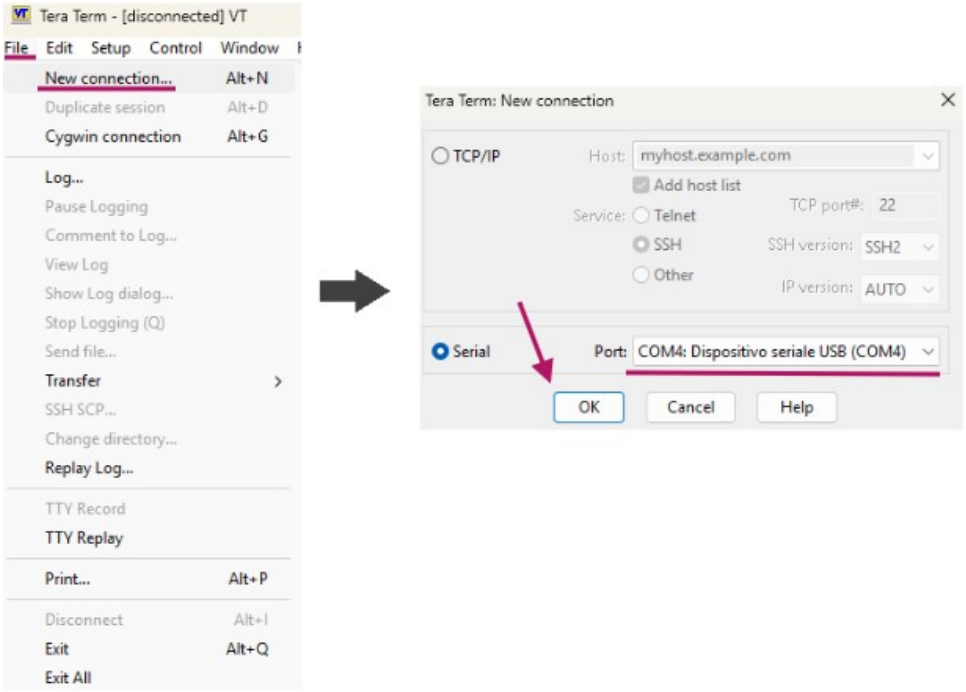
- Select the **Window** appearance and choose your preferred color.



- Save the settings. At the next Tera Term startup, your configuration will be preserved.



- Connect to **COMx**. Remember to select the COM port assigned by Windows for your Arduino board.



- **END**.

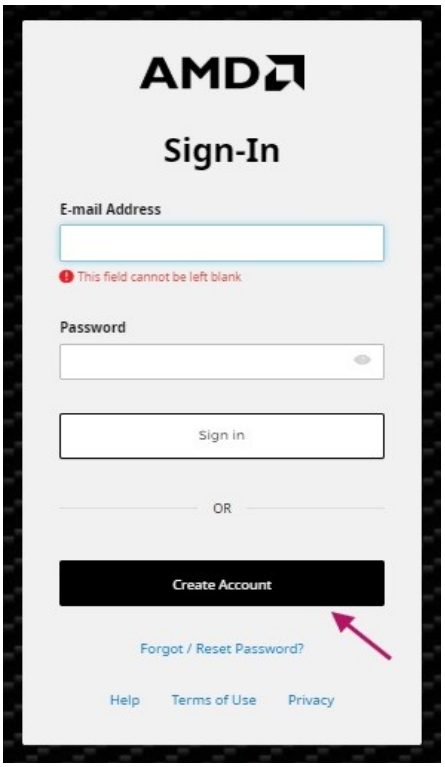
4.4 Vivado 2024.2 Installation on Windows 11

Per poter scaricare **Vivado** è necessario avere un **account AMD**

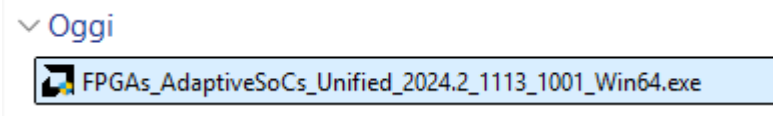
- Go to the website: https://www.xilinx.com/support/download.html

- Click on the appropriate download link.

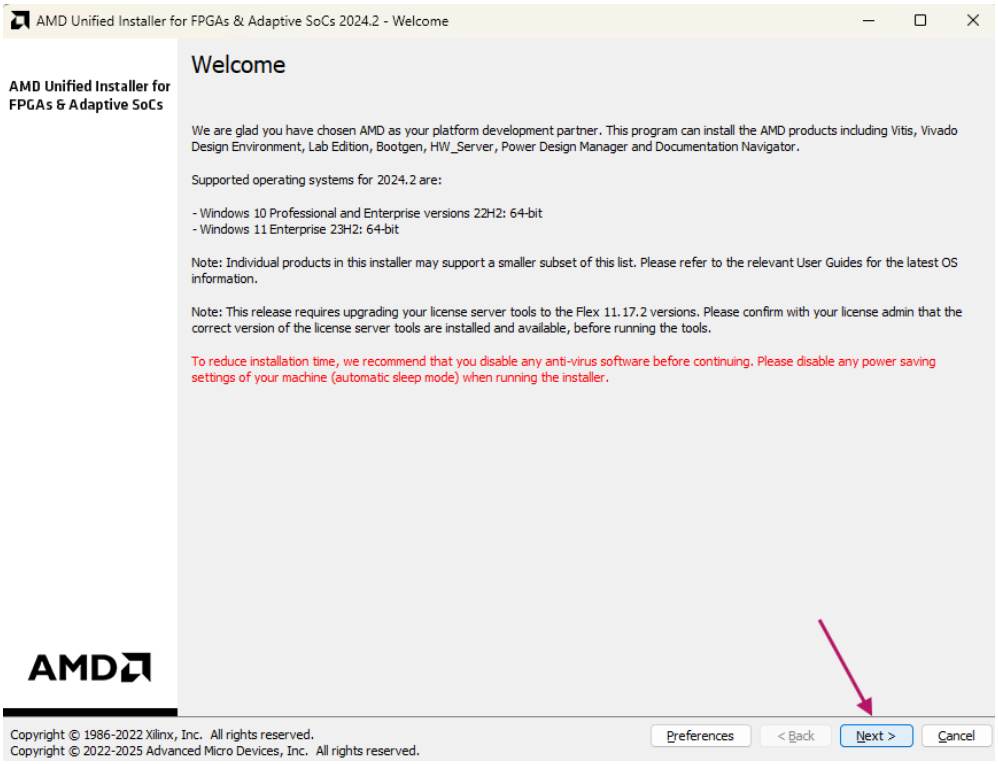
- You will be prompted to register with an email. Complete the registration and the download will start.



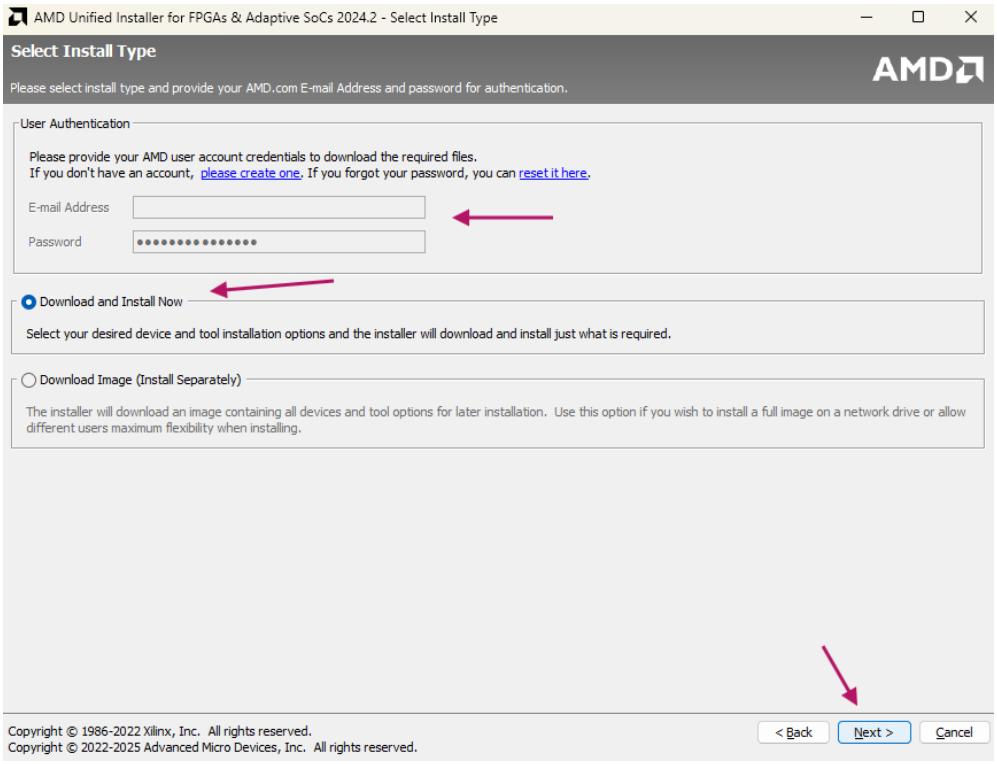
- Launch the downloaded installer.



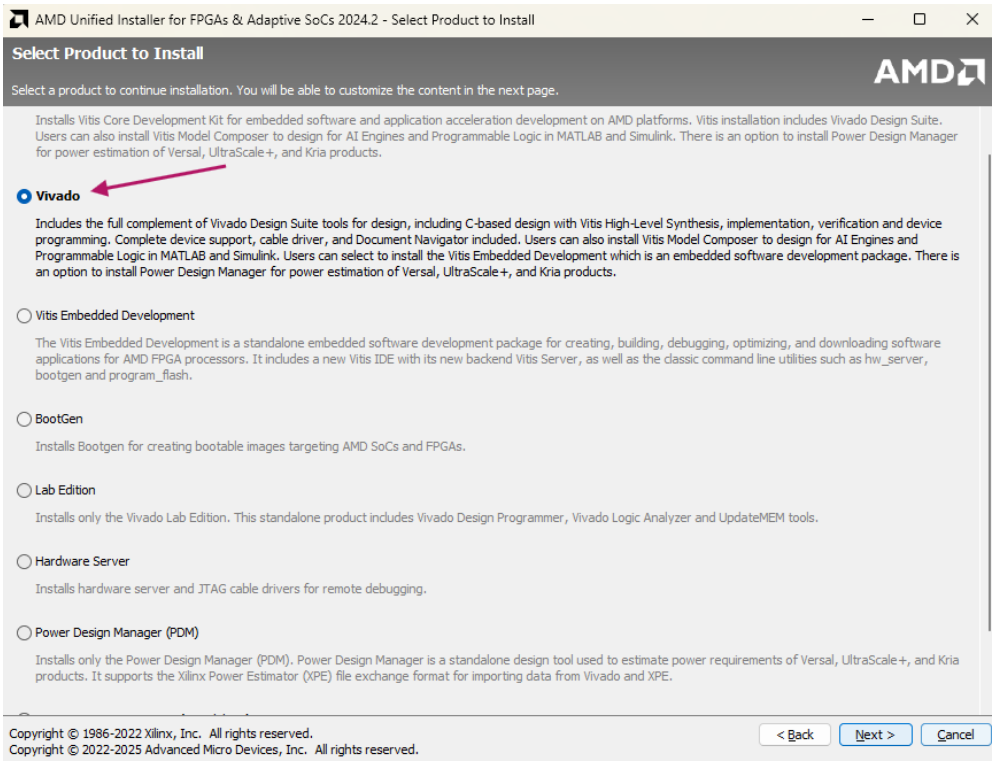
- Press **“NEXT”**.



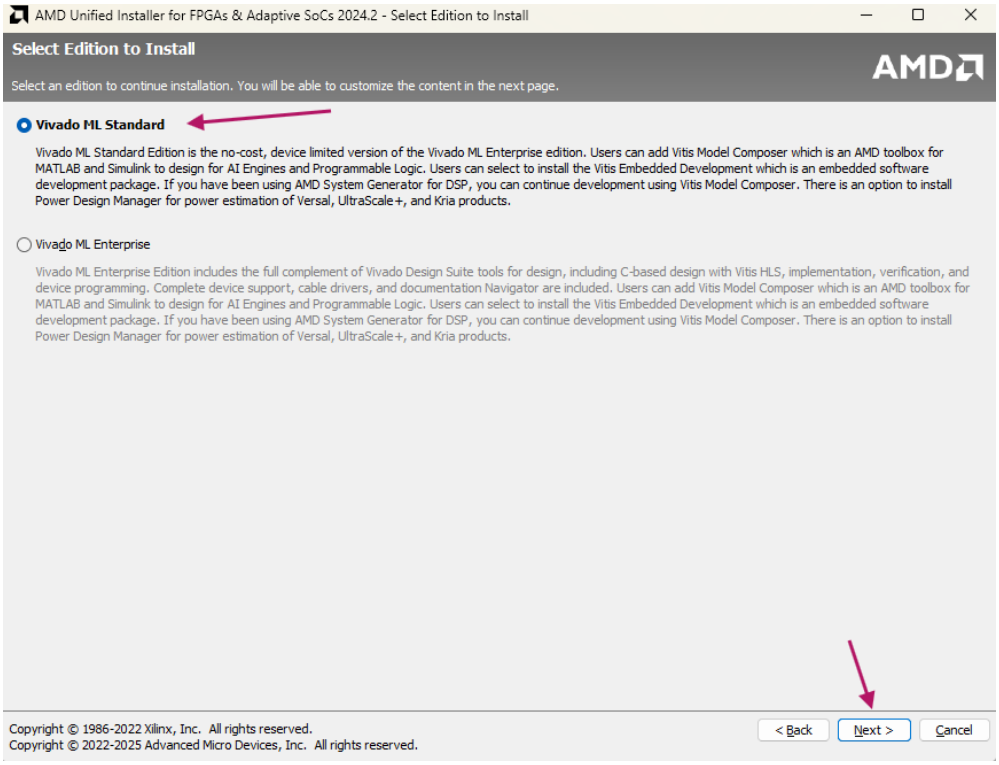
- Enter your **email and password**, then press **“NEXT”**.



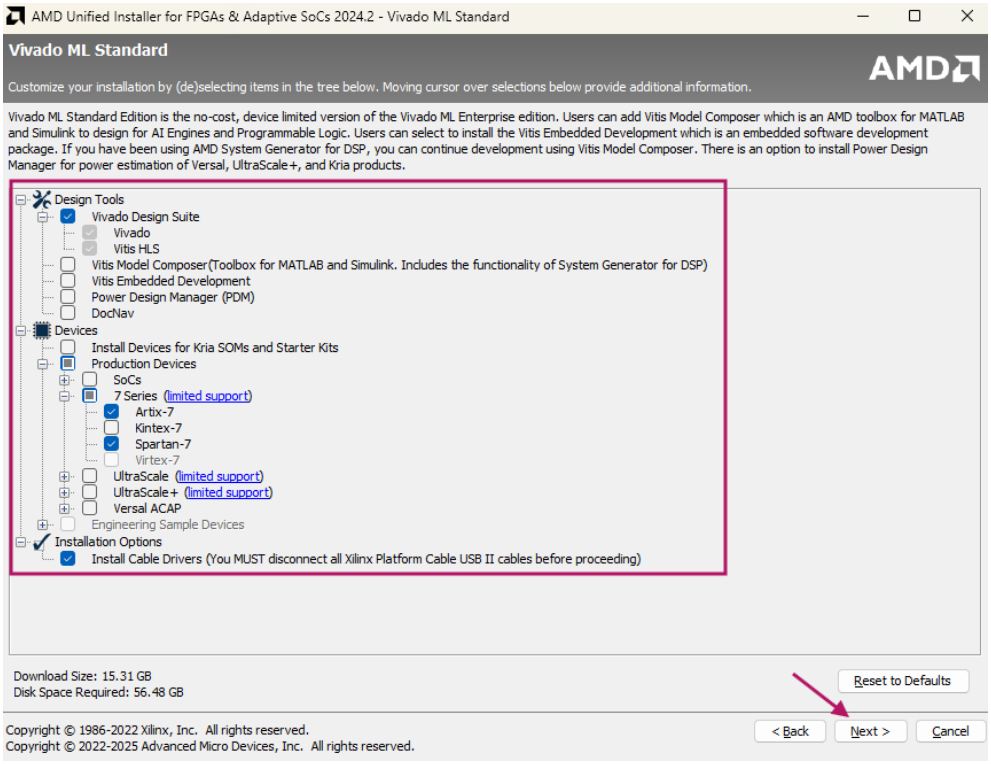
- Select **“Vivado”** and then press **“Next”**.



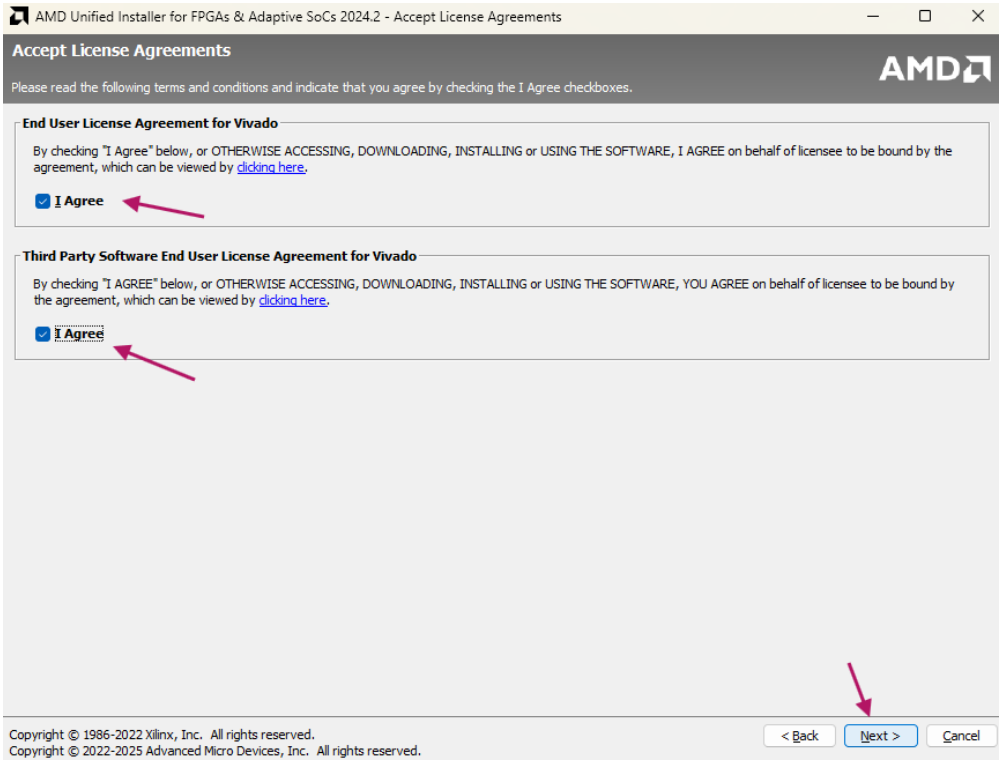
- Select **“Vivado ML Standard”** and then press **“Next”**.



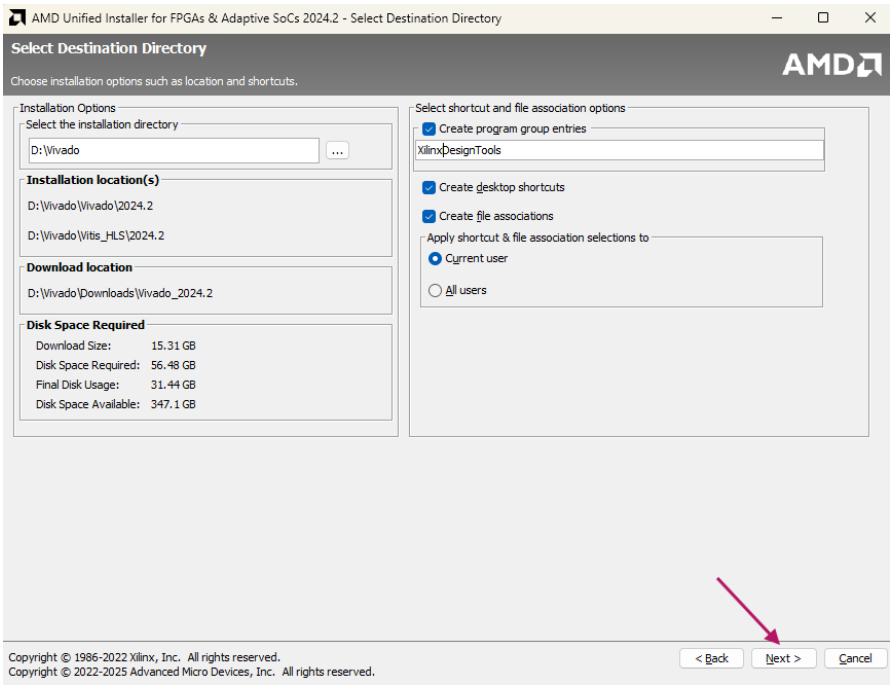
- Check only the packages shown in the red box.



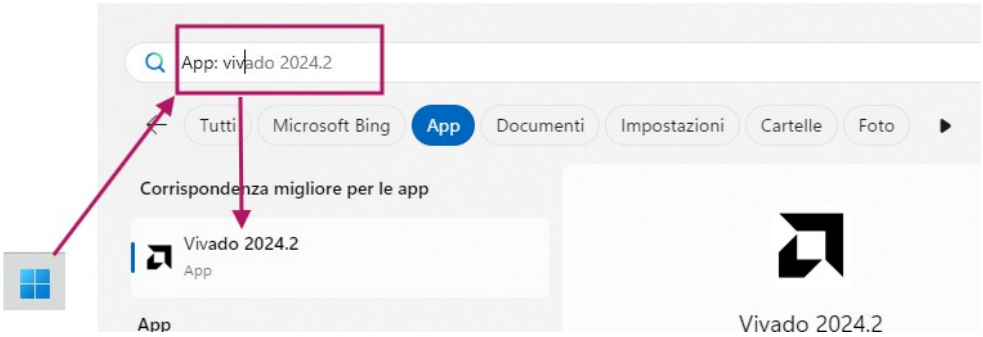
- Agree to the license by checking **“I Agree”** and press **“Next”**.



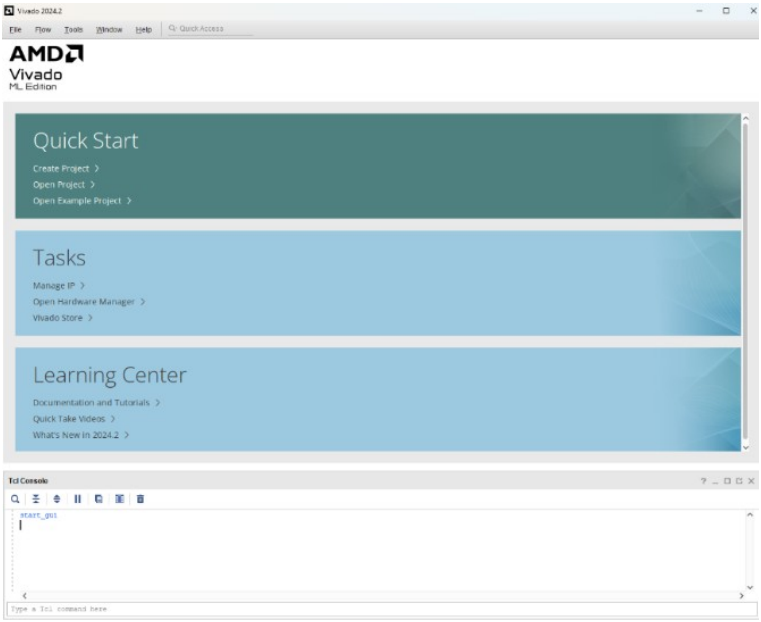
- Select the installation folder and press **“Next”**.



- After the installation is complete, press the **Windows** key, search for **Vivado**, and launch **“Vivado 2024.2”**.



- **Vivado is now ready**.



4.5 Block diagram